

Universidad
Mariana

TALLER ANÁLISIS DE VULNERABILIDADES KALI - LINUX

Sarah Esther Calderón Legarda
Luna Alexandra Beltrán Escobar
Leidy Fernanda Eraso Vásquez

Universidad Mariana
Programa de Ingeniería de Sistemas
Seguridad en Sistemas
Noveno semestre
16 de octubre de 2026

Actividad

Para el desarrollo de esta práctica se toma como entorno de referencia el proyecto de grado **CivicTrackIO**, una aplicación orientada a la seguridad ciudadana que permite el mapeo colaborativo de incidentes y la identificación de zonas de riesgo. A partir de los componentes y servicios con los que cuenta el sistema, se llevan a cabo actividades de revisión y evaluación de seguridad apoyadas en herramientas especializadas, con el fin de detectar posibles vulnerabilidades. Como cierre, se documentan los procedimientos ejecutados, los resultados obtenidos, los riesgos identificados y las recomendaciones necesarias para fortalecer la seguridad del sistema.

Herramientas utilizadas

- **Hydra:** herramienta de auditoría que permite evaluar la solidez de los mecanismos de autenticación mediante intentos de acceso automatizados y controlados.
- **SearchSploit:** herramienta que consulta una base de datos local de vulnerabilidades y exploits conocidos, con el propósito de identificar riesgos potenciales asociados a las tecnologías y versiones empleadas en el proyecto.

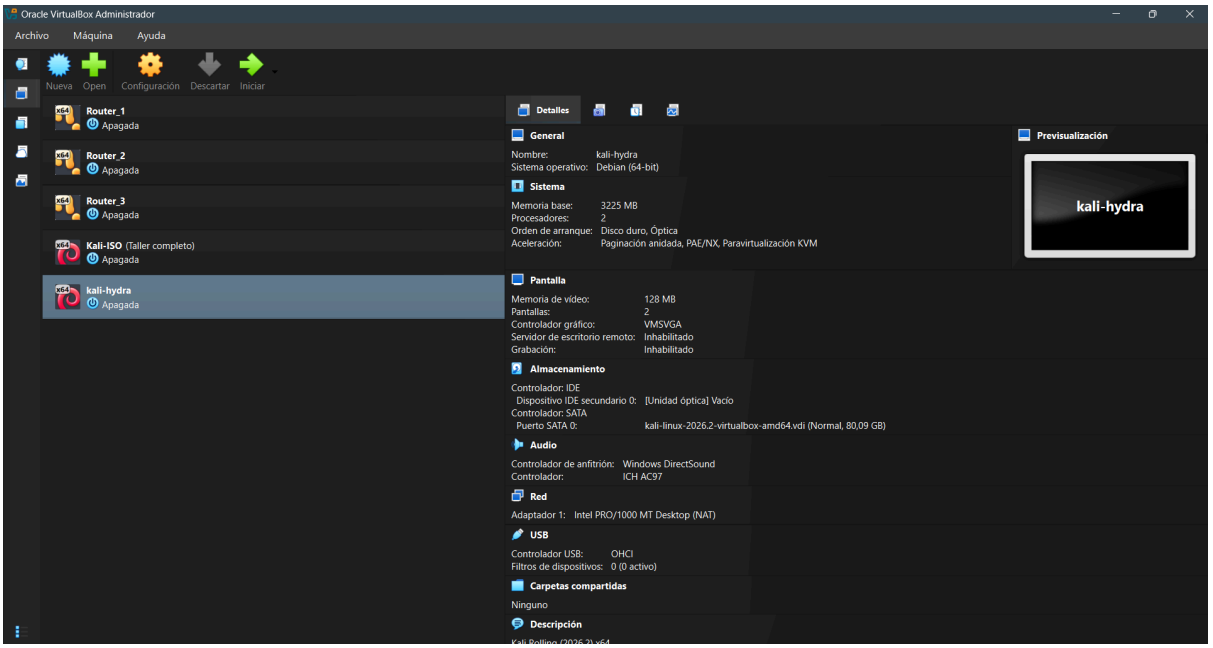
Resultados esperados

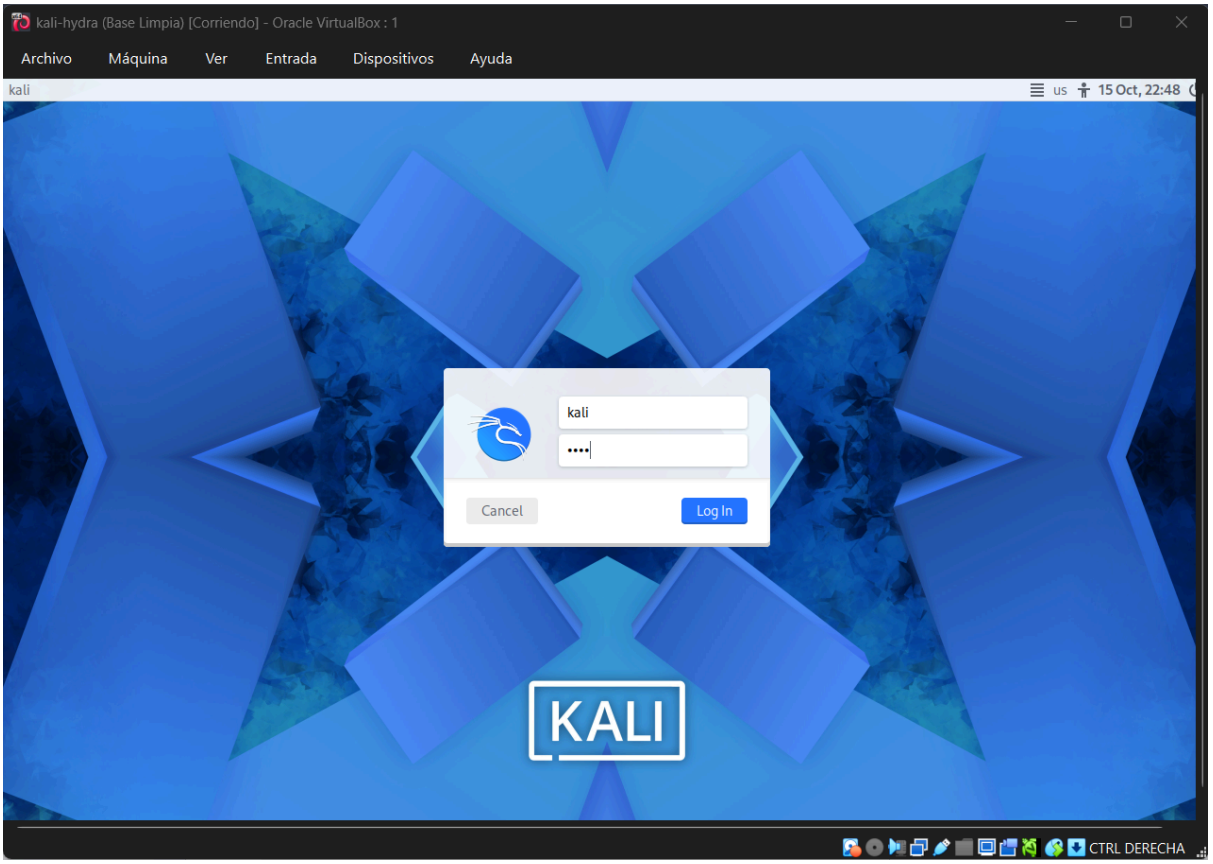
Al finalizar el taller se espera haber identificado posibles debilidades en los servicios y mecanismos de autenticación de **CivicTrackIO**, reconocer las vulnerabilidades conocidas relacionadas con las tecnologías que utiliza y plantear medidas de mitigación que contribuyan a mejorar la seguridad del proyecto.

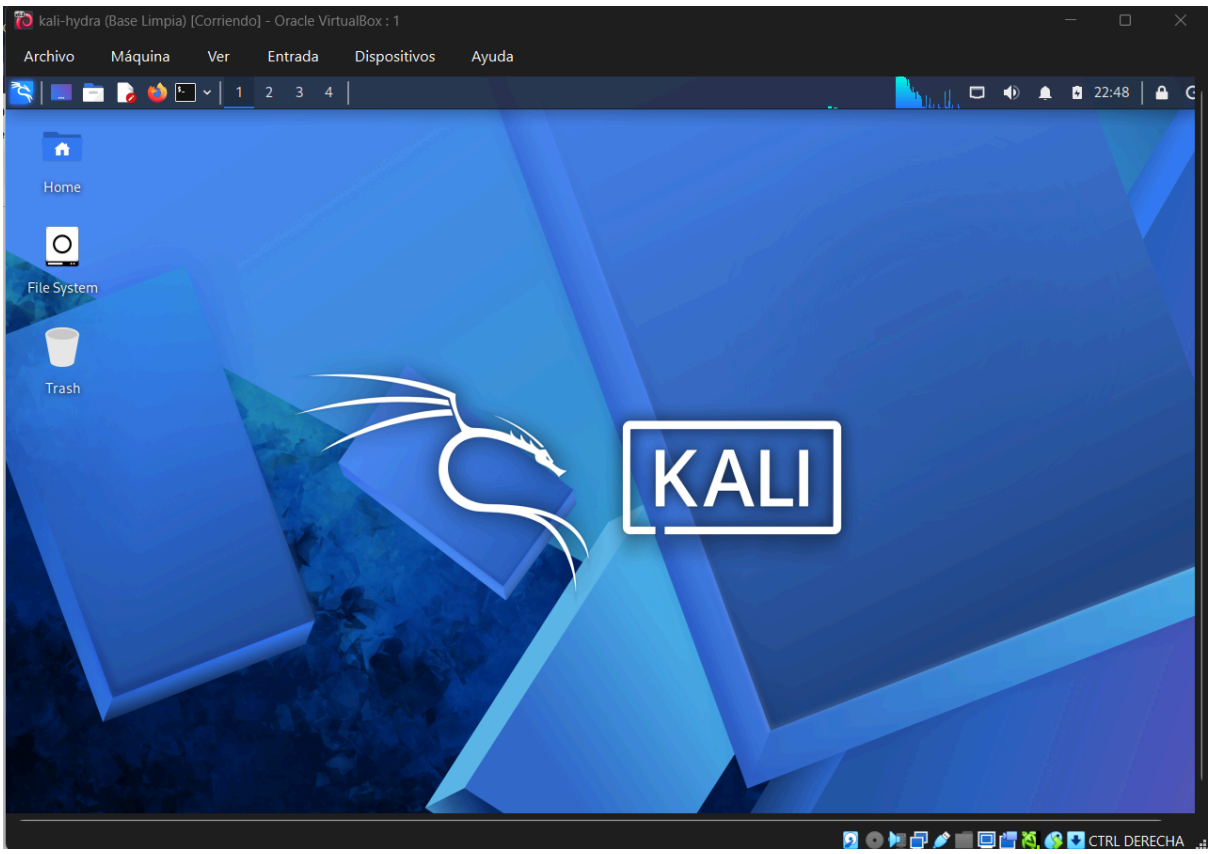
Desarrollo del proyecto

PASO 1: PREPARACIÓN DE LA VM PARA EL TALLER.

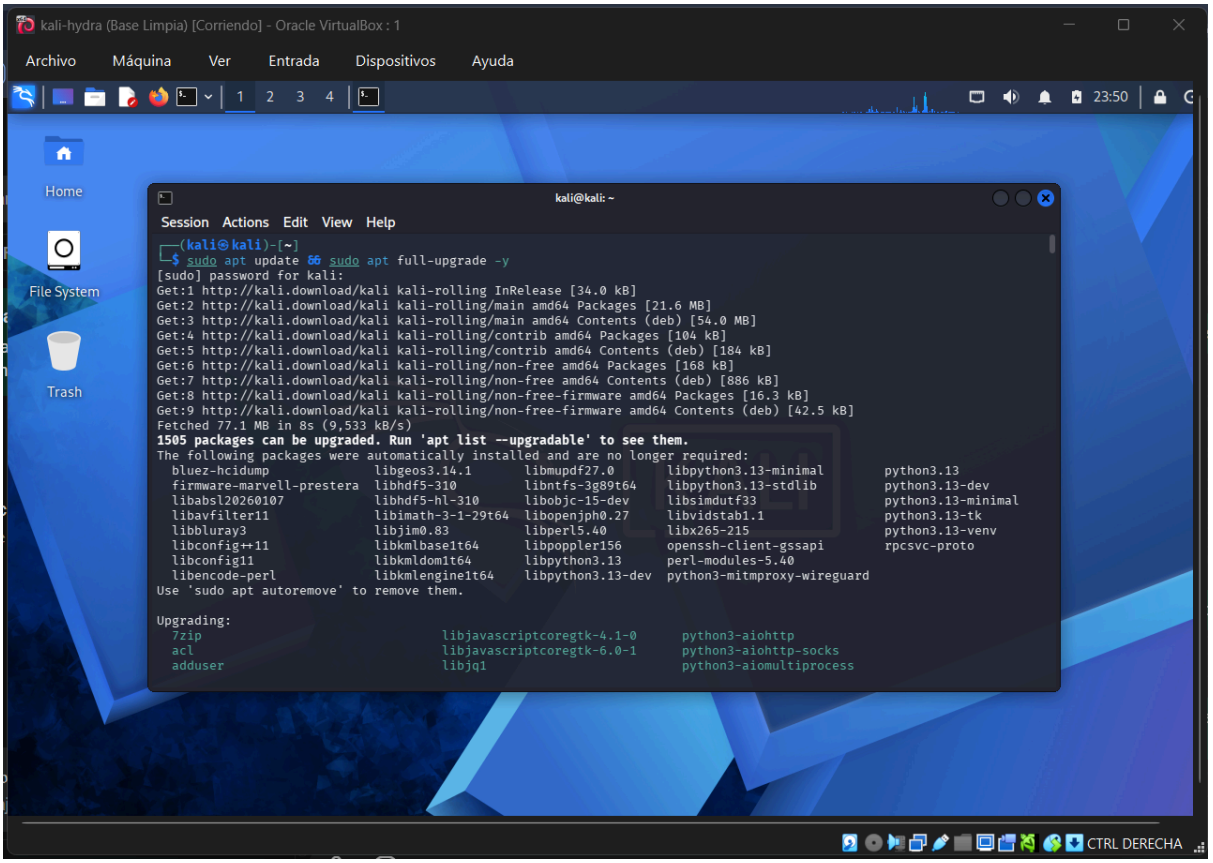
Para comenzar, se instala una máquina virtual preconfigurada de Kali Linux, lo que permite optimizar el tiempo al evitar una instalación desde cero. Se inicia sesión con las configuraciones predefinidas.



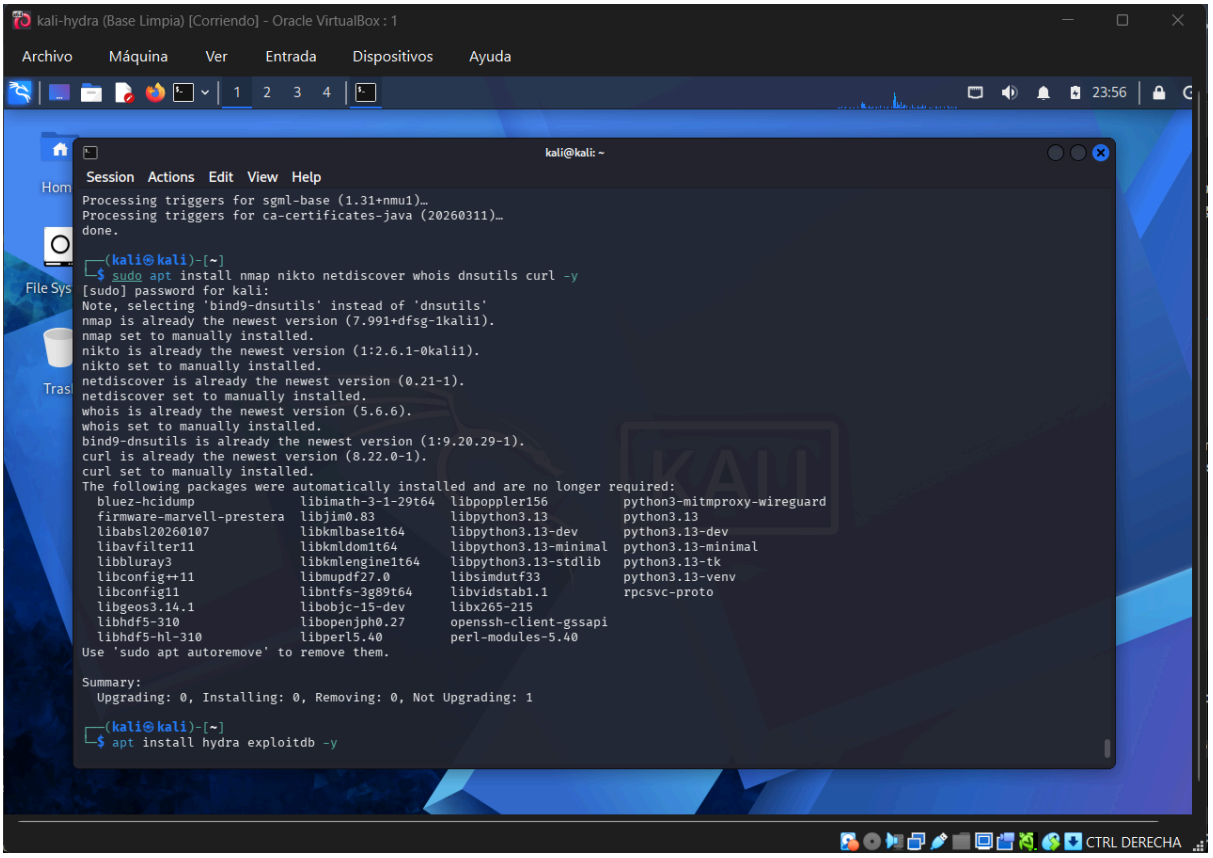


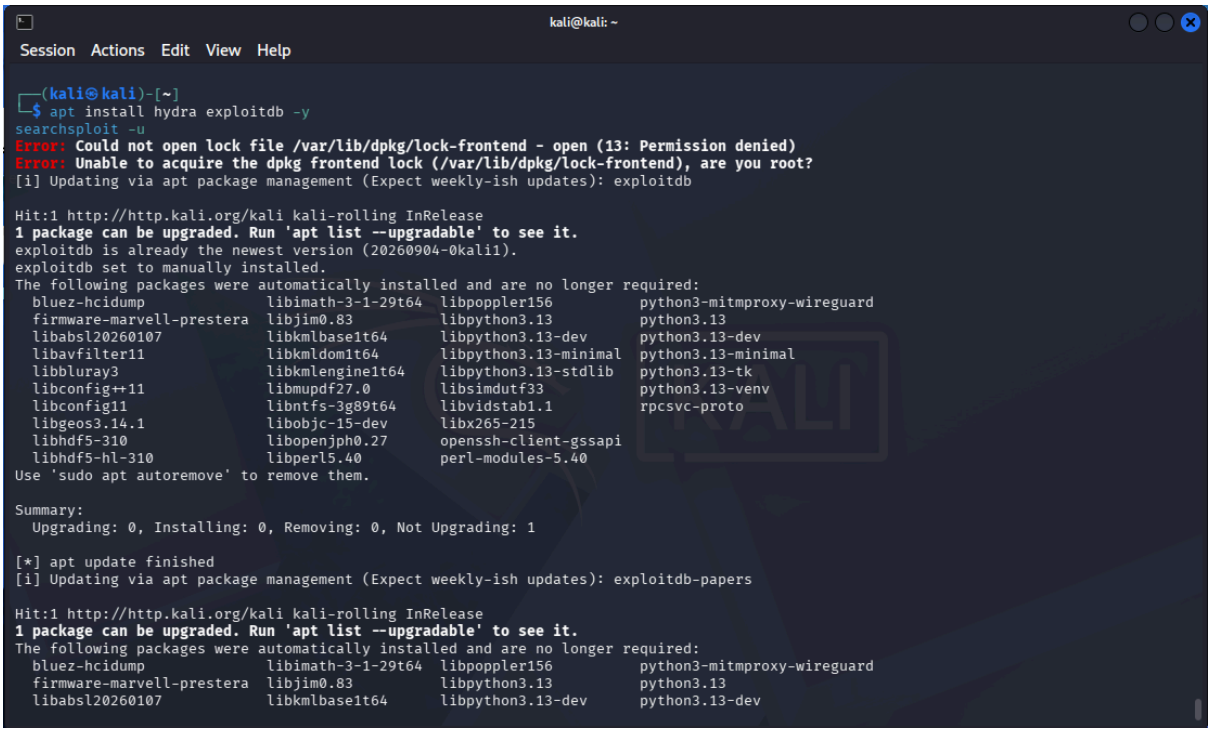


Se actualizan los paquetes de la mv.

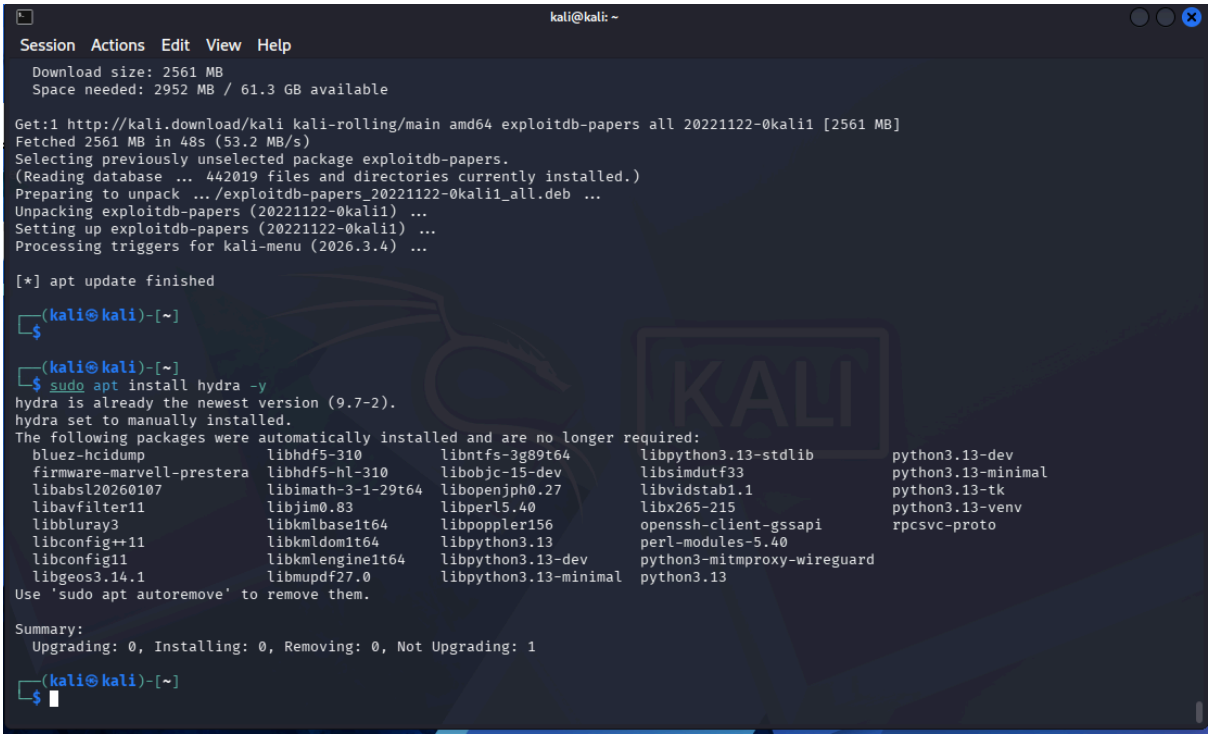


Se instalan las herramientas necesarias para realizar el taller.

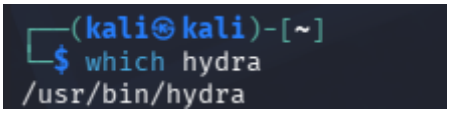




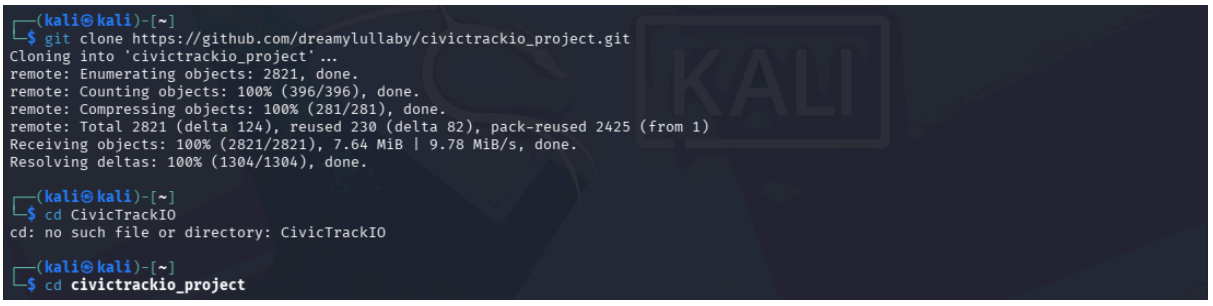
Se produjo un error por un fallo al pegar el comando, eliminado la palabra “sudo”, por lo que no se instaló la herramienta hydra correctamente. Se corrige el comando y se vuelve a correr, instalando correctamente esta vez.



Y se confirma su instalación con *which hydra.*

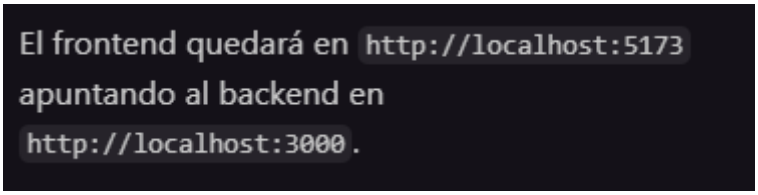


Se crea una rama exclusiva para este taller la cual se llama *taller_hydra,* ya que con el fin de no comprometer la rama *main* solo se harán cambios en este espacio. Una vez creada la rama desde git, se hace un fetch para que pueda ser cargada al repositorio y se clona dentro de la vm.



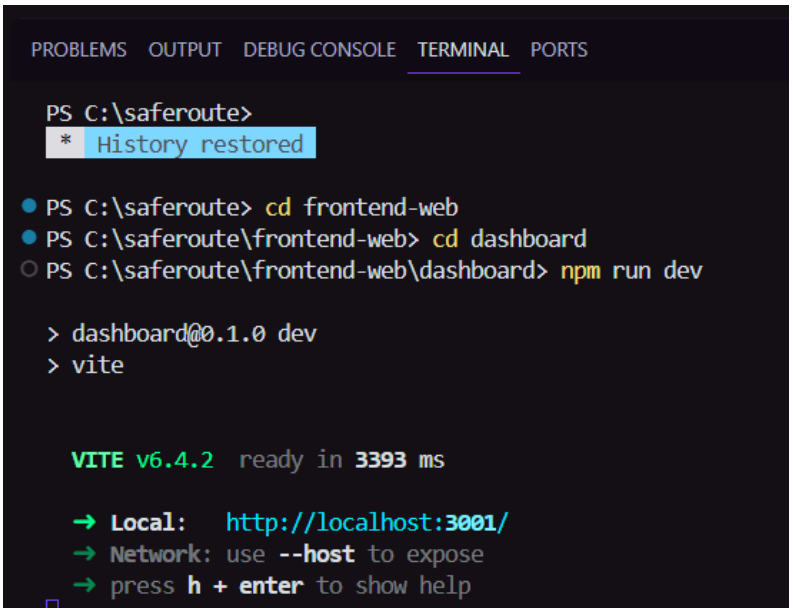


Se actualizan los archivos .env del proyecto local con el fin de no apuntar al enlace público.

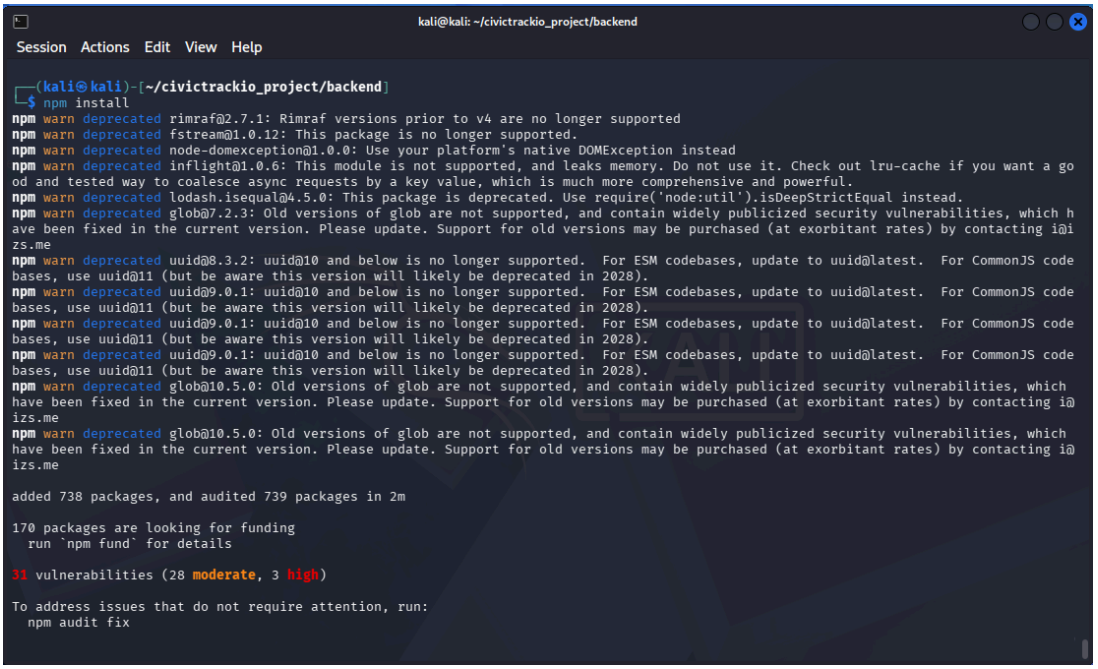


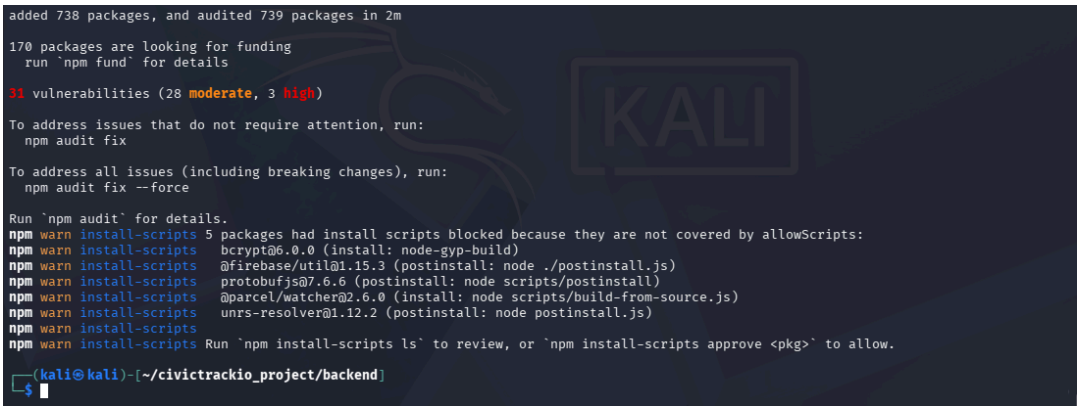
Se corre el proyecto de forma local, como el taller está enfocado en el servicio web, en las pruebas sólo se encontrarán dos pantallas. El frontend relacionado a la parte móvil no se toca. Este se corre directamente en el entorno de programación para verificar si el cambio de puertos fue efectivo.



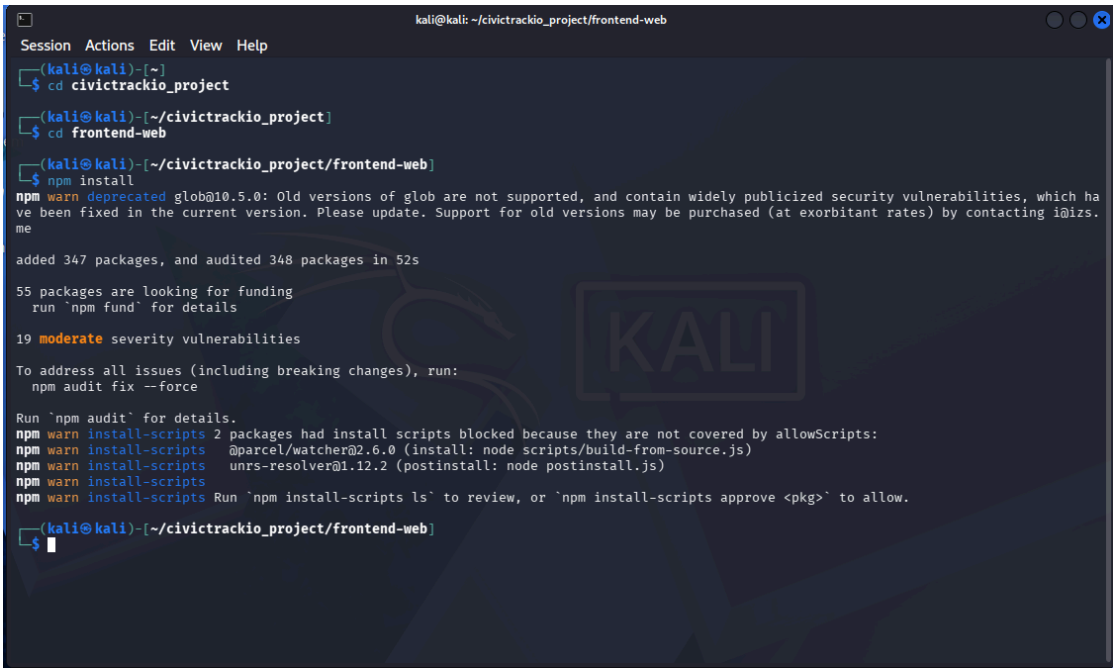


Se instalan las librerías necesarias para correr el proyecto desde nuestra vm. Tanto en backend:





Y en frontend-web:

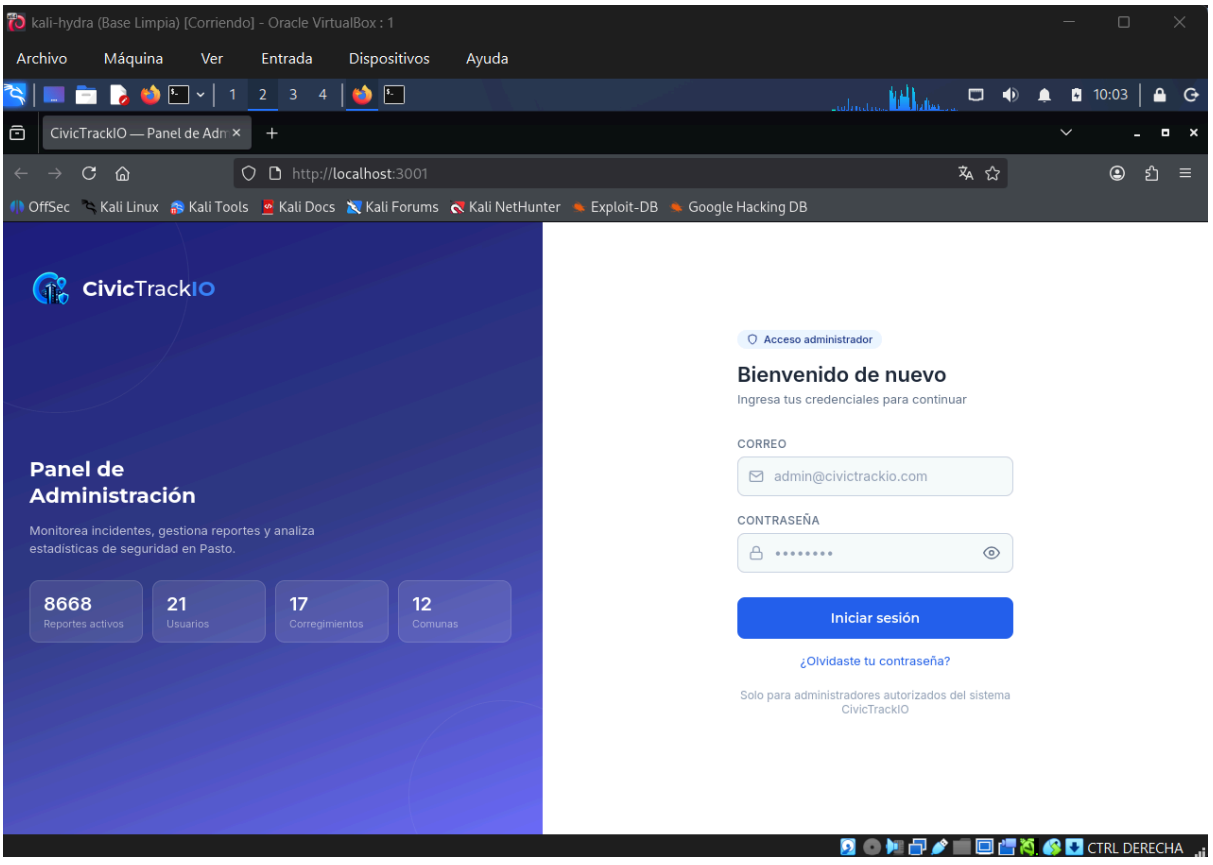


Se corre el proyecto de forma local dentro de la vm.



No sin antes haber configurado las variables de entorno necesarias para lograr la conexión.





Una vez listos los servidores, procedemos a definir el alcance de la prueba, de la siguiente forma:

ALCANCE DE LA PRUEBA — CivicTrackIO (taller Hydra + SearchSploit)

Fecha y hora de ejecución: 16/10/2026, inicio: 9.00 a.m. / finalización:
Ejecutado por: Sarah Calderon
Desarrollado por: Luna Beltrán, Sarah Calderon y Leidy Fernanda Eraso.
Rama de trabajo: taller_hydra

1) Hydra

Objetivo: http://localhost:3000 (backend local, rama taller_hydra)
Endpoint: frontend-web/dashboard/src/page/LoginAdmin.jsx
Cuentas objetivo: labeltran222@umariana.edu.co, lferaso222@umariana.edu.co (admin propias del equipo)
Alcance: únicamente el endpoint de login; sin tocar otras rutas de la API

2) nmap / Nikto

Objetivo: https://civictracio-project.onrender.com, puertos 80/443 Alcance: reconocimiento pasivo/liviano; sin fuerza bruta ni intentos de explotación

3) SearchSploit

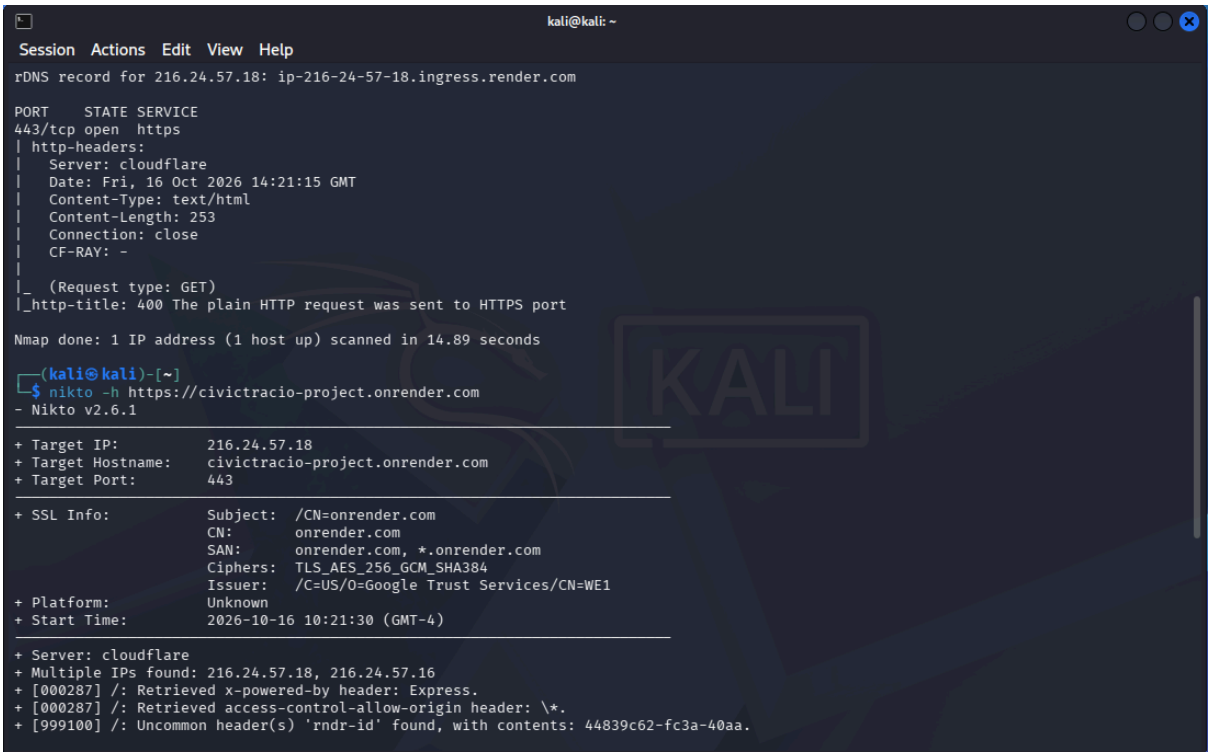
Objetivo: dependencias del proyecto (Express, Node.js, librerías de package.json)
Alcance: consulta local de la base de datos de Exploit-DB; no se ejecuta nada contra ningún host

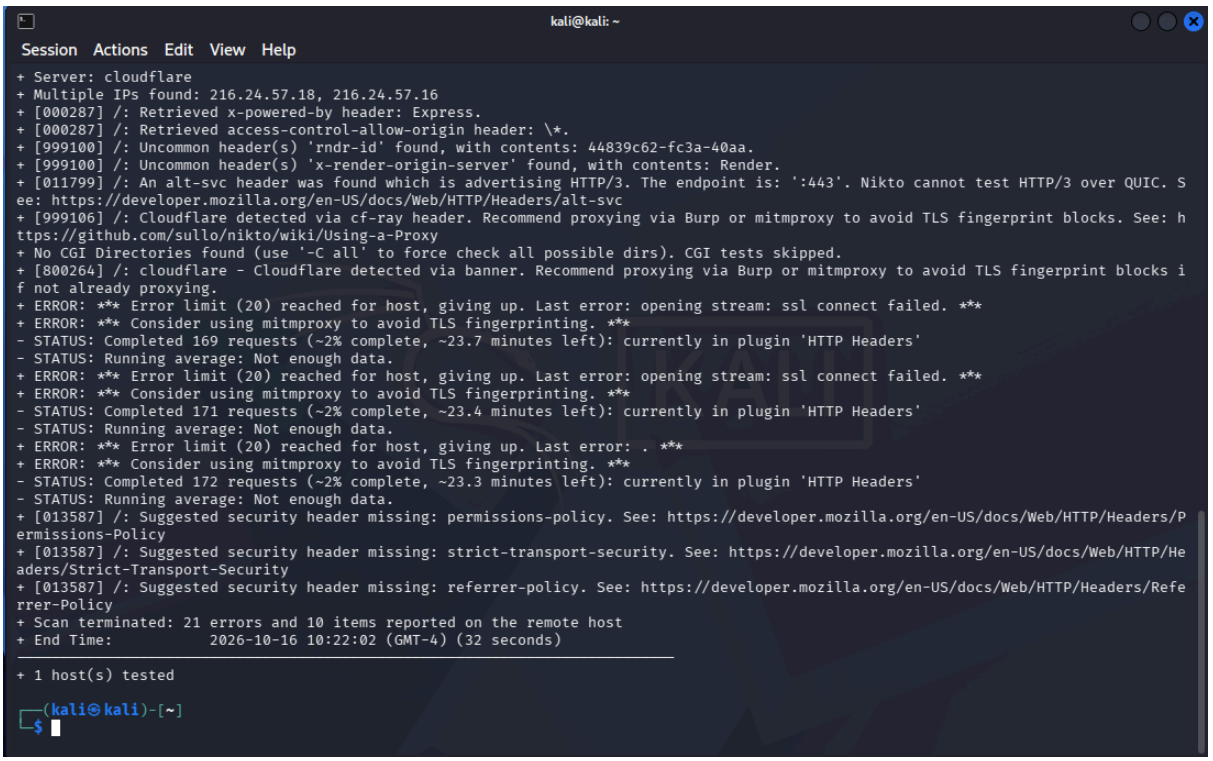
Autorización: proyecto propio (trabajo de grado). Pruebas realizadas únicamente sobre cuentas e infraestructura propias del equipo.

PASO 2: RECONOCIMIENTO CONTRA EL DOMINIO EN RENDER

Se escanean los puertos y el dominio.


kali@kali: ~
Session Actions Edit View Help
┌──(kali㉿kali)-[~]
└─$ host civictracio-project.onrender.com
civictracio-project.onrender.com is an alias for gcp-us-west1-1.origin.onrender.com.
gcp-us-west1-1.origin.onrender.com is an alias for gcp-us-west1-1.origin.onrender.com.cdn.cloudflare.net.
gcp-us-west1-1.origin.onrender.com.cdn.cloudflare.net has address 216.24.57.18
gcp-us-west1-1.origin.onrender.com.cdn.cloudflare.net has address 216.24.57.16
┌──(kali㉿kali)-[~]
└─$ dig +short civictracio-project.onrender.com
gcp-us-west1-1.origin.onrender.com.
gcp-us-west1-1.origin.onrender.com.cdn.cloudflare.net.
216.24.57.16
216.24.57.18
┌──(kali㉿kali)-[~]
└─$ nmap -Pn -p 80,443 civictracio-project.onrender.com
Starting Nmap 7.991 (https://nmap.org) at 2026-10-16 10:20 -0400
Warning: Hostname civictracio-project.onrender.com resolves to 2 IPs. Using 216.24.57.16.
Nmap scan report for civictracio-project.onrender.com (216.24.57.16)
Host is up (0.061s latency).
Other addresses for civictracio-project.onrender.com (not scanned): 216.24.57.18
rDNS record for 216.24.57.16: ip-216-24-57-16.ingress.render.com
PORT STATE SERVICE
80/tcp open http
443/tcp open https
Nmap done: 1 IP address (1 host up) scanned in 0.68 seconds
┌──(kali㉿kali)-[~]
└─$ nmap -Pn -p 443 --script http-title,http-headers civictracio-project.onrender.com
Starting Nmap 7.991 (https://nmap.org) at 2026-10-16 10:21 -0400
Warning: Hostname civictracio-project.onrender.com resolves to 2 IPs. Using 216.24.57.18.
Nmap scan report for civictracio-project.onrender.com (216.24.57.18)
Host is up (0.061s latency).
Other addresses for civictracio-project.onrender.com (not scanned): 216.24.57.16
rDNS record for 216.24.57.18: ip-216-24-57-18.ingress.render.com


kali@kali: ~
Session Actions Edit View Help
rDNS record for 216.24.57.18: ip-216-24-57-18.ingress.render.com
PORT STATE SERVICE
443/tcp open https
| http-headers:
| Server: cloudflare
| Date: Fri, 16 Oct 2026 14:21:15 GMT
| Content-Type: text/html
| Content-Length: 253
| Connection: close
| CF-RAY: -
|
|_ (Request type: GET)
|_http-title: 400 The plain HTTP request was sent to HTTPS port
Nmap done: 1 IP address (1 host up) scanned in 14.89 seconds
┌──(kali㉿kali)-[~]
└─$ nikto -h https://civictracio-project.onrender.com
- Nikto v2.6.1
+ Target IP: 216.24.57.18
+ Target Hostname: civictracio-project.onrender.com
+ Target Port: 443
+ SSL Info: Subject: /CN=onrender.com
CN: onrender.com
SAN: onrender.com, *.onrender.com
Ciphers: TLS_AES_256_GCM_SHA384
Issuer: /C=US/O=Google Trust Services/CN=WE1
+ Platform: Unknown
+ Start Time: 2026-10-16 10:21:30 (GMT-4)
+ Server: cloudflare
+ Multiple IPs found: 216.24.57.18, 216.24.57.16
+ [000287] /: Retrieved x-powered-by header: Express.
+ [000287] /: Retrieved access-control-allow-origin header: *.
+ [999100] /: Uncommon header(s) 'rndr-id' found, with contents: 44839c62-fc3a-40aa.



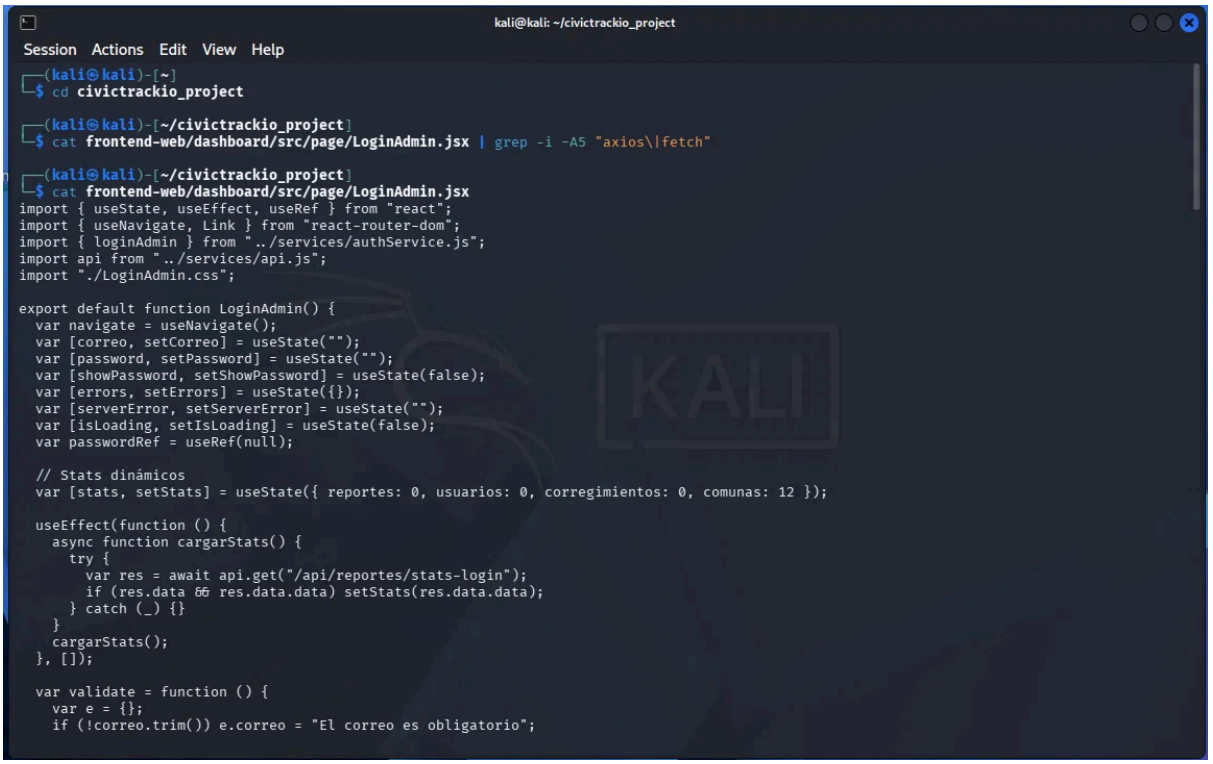
Con los resultados obtenidos en esta primera fase de reconocimiento, se puede afirmar que CivicTrackIO no presenta hasta el momento vulnerabilidades críticas o explotables de forma directa. El uso de Cloudflare como capa intermedia frente al dominio de Render es un aspecto positivo, pues añade una protección adicional antes de que el tráfico llegue al servidor real. Sin embargo, el escaneo evidencia que el sitio expone más información de la necesaria sobre su propia infraestructura: se filtra la tecnología del backend (Express), el proveedor de hosting (Render) y se mantiene el CORS abierto a cualquier origen.

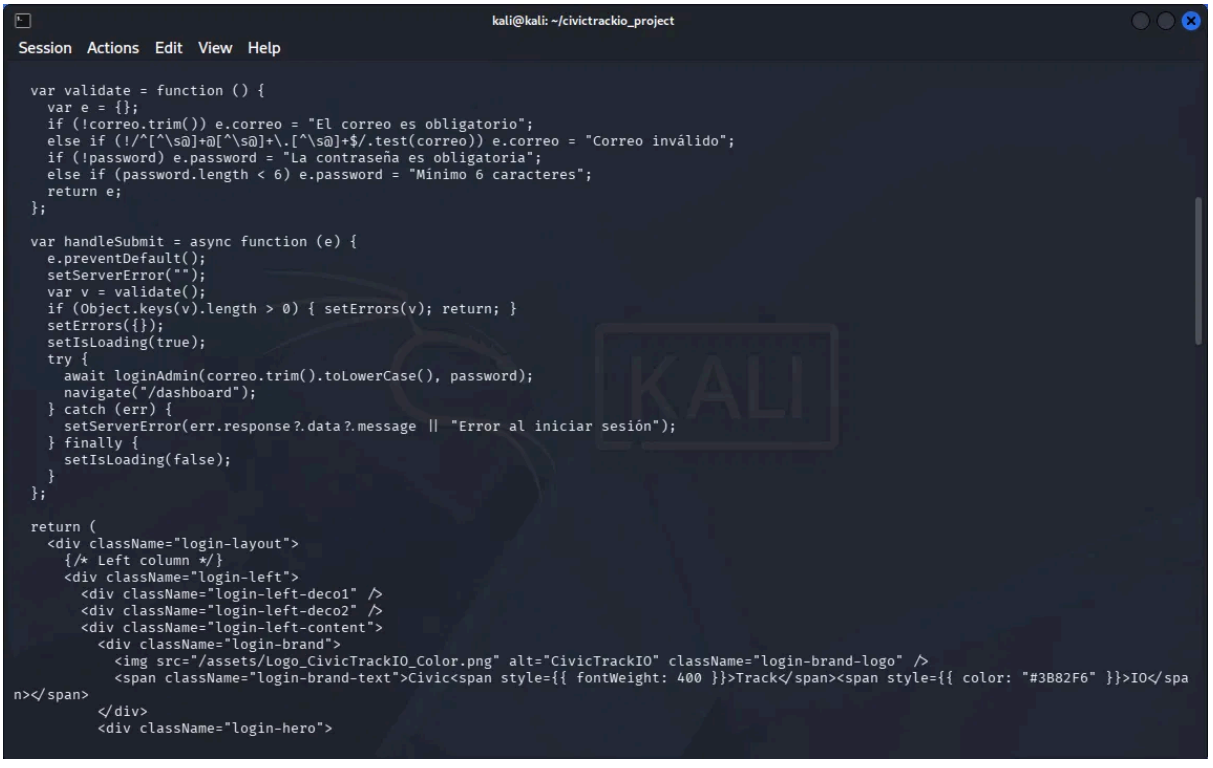
Esto no permite un acceso no autorizado por sí solo, pero facilita que un atacante identifique por dónde buscar vulnerabilidades específicas. Adicionalmente, se identificó la ausencia de varias cabeceras de seguridad recomendadas (HSTS, Permissions-Policy, Referrer-Policy), consideradas una buena práctica estándar en aplicaciones en producción. En conjunto, estos hallazgos ubican al proyecto en un nivel de seguridad intermedio: la infraestructura base está razonablemente protegida, pero existen descuidos de configuración que deben corregirse. Las siguientes fases del taller, correspondientes a Hydra y SearchSploit, permitirán determinar con mayor precisión si estos hallazgos representan un riesgo real para el sistema.

PASO 3: IDENTIFICAR EL MECANISMO DEL LOGIN.

Antes de configurar Hydra, se procede a confirmar la ruta exacta del login y el formato del cuerpo de la petición, en vez de asumir valores genéricos. Se inicia revisando el componente LoginAdmin.jsx,

donde se identifica que la autenticación no se hace directamente con axios/fetch dentro del componente, sino a través de una función loginAdmin(correo, password) importada del servicio authService.js. También se confirma que los campos del formulario corresponden a correo y password, y que el correo se normaliza (trim() y toLowerCase()) antes de enviarse.





kali@kali: ~/civictrackio_project

Session Actions Edit View Help

```
  return (
    <div className="login-layout">
      {/* Left column */}
      <div className="login-left">
        <div className="login-left-deco1" />
        <div className="login-left-deco2" />
        <div className="login-left-content">
          <div className="login-brand">
            <img src="/assets/Logo_CivicTrackIO_Color.png" alt="CivicTrackIO" className="login-brand-logo" />
            <span className="login-brand-text">Civic<span style={{ fontWeight: 400 }}>Track</span><span style={{ color: "#3B82F6" }}>IO</spa
n></span>
          </div>
          <div className="login-hero">
            <h1 className="login-hero-title">{"Panel de\nAdministración"}</h1>
            <p className="login-hero-sub">Monitorea incidentes, gestiona reportes y analiza estadísticas de seguridad en Pasto.</p>
            <div className="login-stats">
              <StatCard target={stats.reportes} label="Reportes activos" />
              <StatCard target={stats.usuarios} label="Usuarios" />
              <StatCard target={stats.corregimientos} label="Corregimientos" />
              <StatCard target={stats.comunas} label="Comunas" />
            </div>
          </div>
        </div>
      </div>

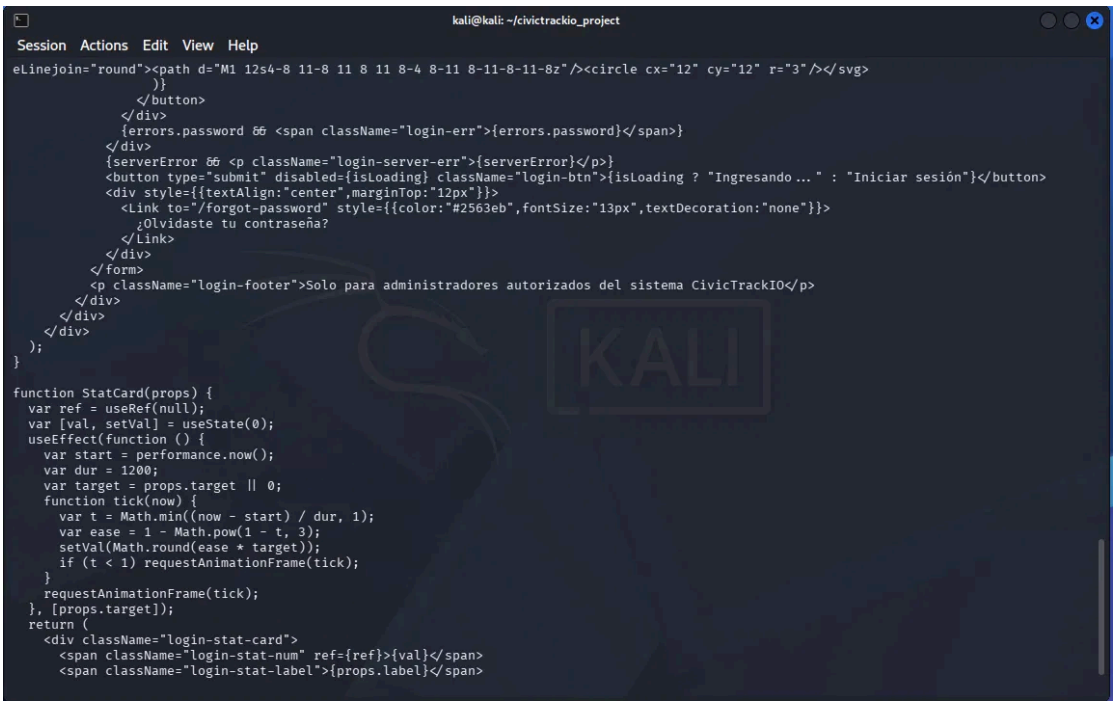
      {/* Right column */}
      <div className="login-right">
        <div className="login-form-wrap">
          <div className="login-badge-admin">
            <svg width="12" height="12" viewBox="0 0 24 24" fill="none" stroke="#1E3A8A" strokeWidth="2"><path d="M12 22s8-4 8-10V5l-8-3-8 3
v7c0 6 8 10 8 10z"/></svg>
            <span>Acceso administrador</span>
          </div>
          <h2 className="login-form-title">Bienvenido de nuevo</h2>
          <p className="login-form-sub">Ingresa tus credenciales para continuar</p>

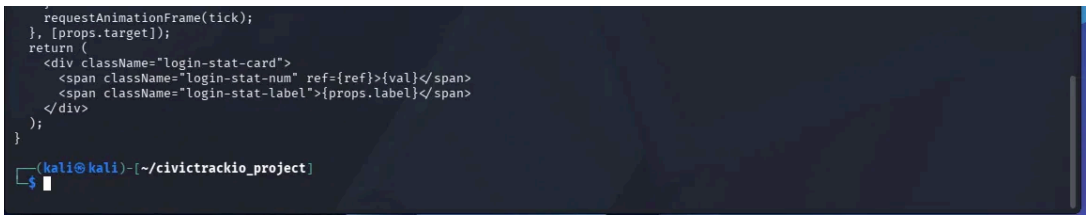
          <form onSubmit={handleSubmit} noValidate>
            <div className="login-field">
              <label htmlFor="correo">CORREO</label>
              <div className="login-input-wrap">
```

kali@kali: ~/civictrackio_project

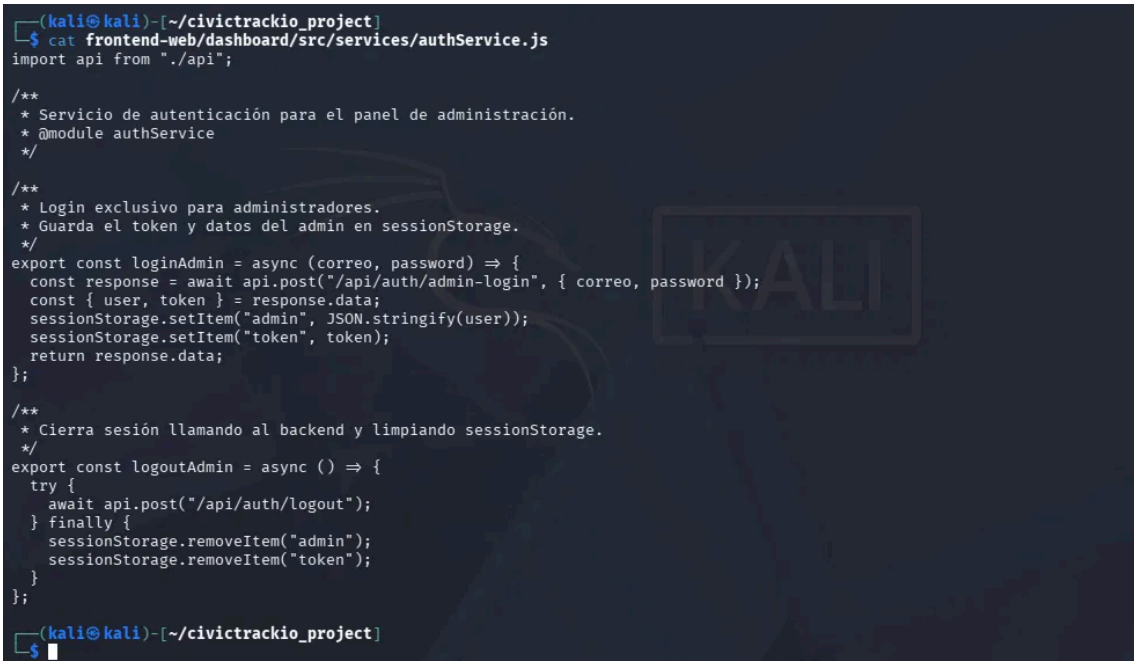
Session Actions Edit View Help

```
          <form onSubmit={handleSubmit} noValidate>
            <div className="login-field">
              <label htmlFor="correo">CORREO</label>
              <div className="login-input-wrap">
                <svg className="login-input-icon" width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="#94A3B8" strokeWidth="2"><p
ath d="M4 4h16c1.1 0 2 .9 2 2v12c0 1.1-.9 2-2 2H4c-1.1 0-2-.9-2-2V6c0-1.1.9-2 2-2z"/><polyline points="22,6 12,13 2,6"/></svg>
                <input
                  id="correo"
                  type="email"
                  value={correo}
                  onChange={function (e) { setCorreo(e.target.value); }}
                  onKeyDown={function (e) { if (e.key === "Enter") { e.preventDefault(); passwordRef.current?.focus(); } }}
                  placeholder="admin@civictrackio.com"
                  className={errors.correo ? "login-input login-input-err" : "login-input"}
                />
              </div>
              {errors.correo && <span className="login-err">{errors.correo}</span>}
            </div>
            <div className="login-field">
              <label htmlFor="password">CONTRASEÑA</label>
              <div className="login-input-wrap">
                <svg className="login-input-icon" width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="#94A3B8" strokeWidth="2"><r
ect x="3" y="11" width="18" height="11" rx="2"/><path d="M7 11V7a5 5 0 0 1 10 0v4"/></svg>
                <input
                  id="password"
                  ref={passwordRef}
                  type={showPassword ? "text" : "password"}
                  value={password}
                  onChange={function (e) { setPassword(e.target.value); }}
                  placeholder="••••••••"
                  className={errors.password ? "login-input login-input-err login-input-password" : "login-input login-input-password"}
                />
                <button type="button" className="login-eye-btn" onClick={function () { setShowPassword(!showPassword); }} tabIndex={-1} aria
-label={showPassword ? "Ocultar contraseña" : "Mostrar contraseña"}>
                  {showPassword ? (
                    <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="#64748B" strokeWidth="2" strokeLinecap="round" strok
eLinejoin="round"><path d="M17.94 17.94A10.07 10.07 0 0 1 12 20c-7 0-11-8-11-8a18.45 18.45 0 0 1 5.06-5.94"/><path d="M9.9 4.24A9.12 9.12 0
0 1 12 4c7 0 11 8 11 8a18.5 18.5 0 0 1-2.16 3.19"/><line x1="1" y1="1" x2="23" y2="23"/></svg>
                  ) : (
                    <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="#64748B" strokeWidth="2" strokeLinecap="round" strok
```





Para precisar la petición real, se revisan los archivos authService.js y api.js. En authService.js se encuentra la llamada exacta: api.post("/api/auth/admin-login", { correo, password }). En api.js se confirma que el cliente HTTP es Axios, preconfigurado con baseURL: http://localhost:3000, y que inyecta automáticamente el token JWT en cada petición mediante un interceptor.



```
┌──(kali㉿kali)-[~/civictrackio_project]
└─$ cat frontend-web/dashboard/src/services/api.js
/**
 * Cliente HTTP Axios preconfigurado para el panel de administración.
 * Inyecta automáticamente el token JWT en cada petición.
 * @module api
 */
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:3000",
});

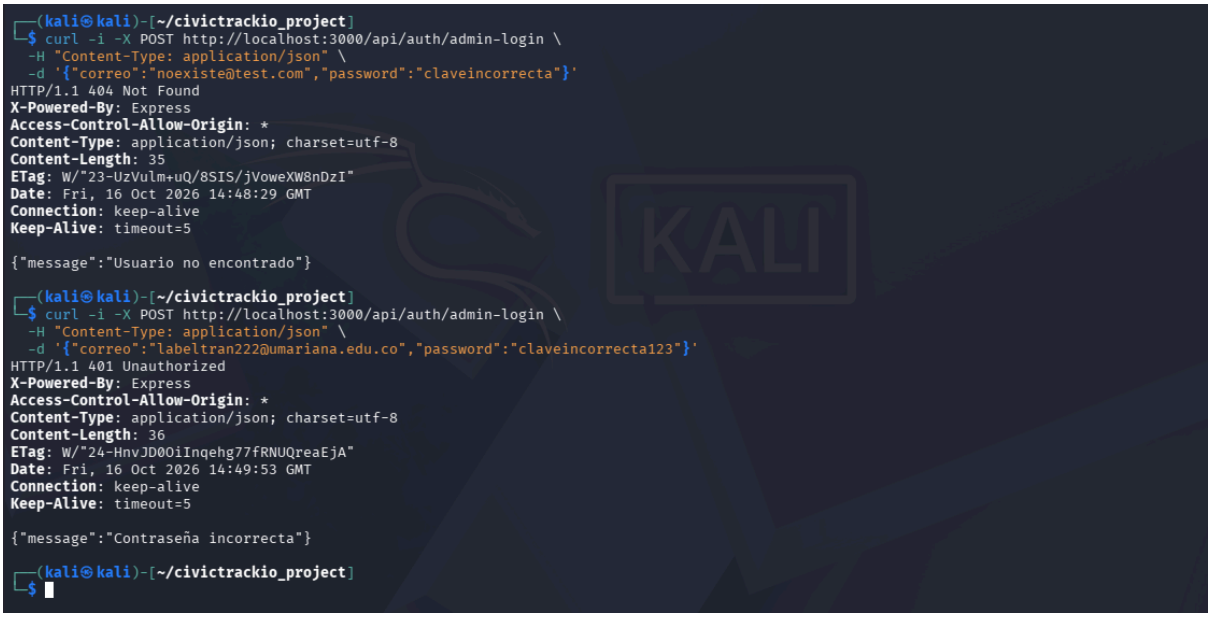
// Agrega el token JWT en cada request si existe
api.interceptors.request.use((config) => {
  const token = sessionStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default api;

┌──(kali㉿kali)-[~/civictrackio_project]
└─$
```

Con la ruta y los nombres de los campos ya identificados en el frontend, se confirma el comportamiento real contra el backend corriendo en local. Primero se localiza el controlador correspondiente con grep sobre backend/src, encontrando la ruta POST /api/auth/admin-login en userRoutes.js, manejada por la función loginAdmin en userController.js. Luego se realizan dos pruebas con curl para confirmar los mensajes de error reales del servidor: una con un correo inexistente, que responde 404 Not Found con el mensaje "Usuario no encontrado"; y otra con una de las cuentas admin reales del equipo pero contraseña incorrecta, que responde 401 Unauthorized con el mensaje "Contraseña incorrecta". Este segundo caso es el relevante para Hydra, ya que será la respuesta que se repita en la mayoría de los intentos fallidos del diccionario.

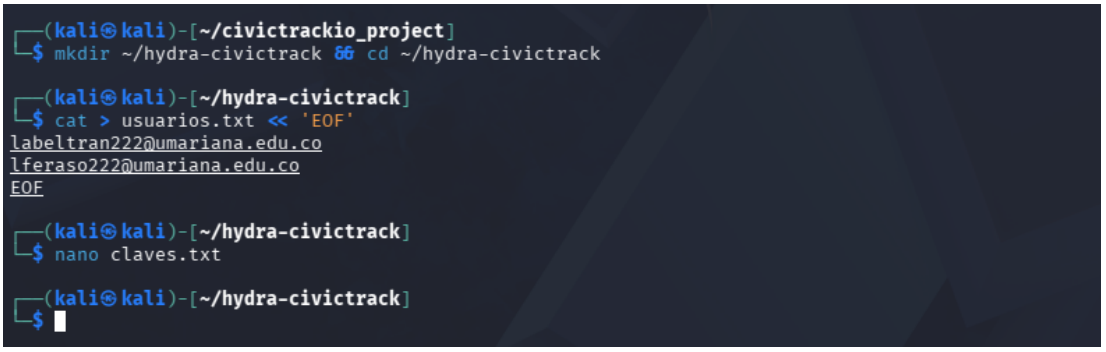
```
┌──(kali㉿kali)-[~/civictrackio_project]
└─$ cd ~/civictrackio_project
grep -ri -B2 -A10 "admin-login" backend/src --include="*.js"
backend/src/interfaces/routes/userRoutes.js-router.post("/google/check", checkGoogleUser);
backend/src/interfaces/routes/userRoutes.js-
backend/src/interfaces/routes/userRoutes.js:/** POST /api/auth/admin-login — Login exclusivo para administradores */
backend/src/interfaces/routes/userRoutes.js:router.post("/admin-login", loginAdmin);
backend/src/interfaces/routes/userRoutes.js-
backend/src/interfaces/routes/userRoutes.js-/** POST /api/auth/logout — Cierra sesión (requiere token válido) */
backend/src/interfaces/routes/userRoutes.js-router.post("/logout", authenticate, logoutUser);
backend/src/interfaces/routes/userRoutes.js-
backend/src/interfaces/routes/userRoutes.js-/** PATCH /api/auth/username — Actualiza el apodo del usuario (requiere token válido) */
backend/src/interfaces/routes/userRoutes.js-router.patch("/username", authenticate, updateUsername);
backend/src/interfaces/routes/userRoutes.js-
backend/src/interfaces/routes/userRoutes.js-/** PATCH /api/auth/fcm-token — Guarda el FCM token del dispositivo (requiere token válido) */
backend/src/interfaces/routes/userRoutes.js-router.patch("/fcm-token", authenticate, updateFcmToken);
backend/src/interfaces/routes/userRoutes.js-
--
backend/src/interfaces/controllers/userController.js-
backend/src/interfaces/controllers/userController.js-/**
backend/src/interfaces/controllers/userController.js: * Maneja POST /api/auth/admin-login
backend/src/interfaces/controllers/userController.js- * Login exclusivo para administradores. Verifica rol antes de responder.
backend/src/interfaces/controllers/userController.js- * Acepta correo o username (case-insensitive).
backend/src/interfaces/controllers/userController.js- * @param {import('express').Request} req - Body: { correo, password }
backend/src/interfaces/controllers/userController.js- * @param {import('express').Response} res
backend/src/interfaces/controllers/userController.js- */
backend/src/interfaces/controllers/userController.js-export const loginAdmin = async (req, res) => {
backend/src/interfaces/controllers/userController.js-
backend/src/interfaces/controllers/userController.js-  try {
backend/src/interfaces/controllers/userController.js-
backend/src/interfaces/controllers/userController.js-    const { correo, password } = req.body;
```



En la siguiente tabla se resume la información encontrada a lo largo de este paso:

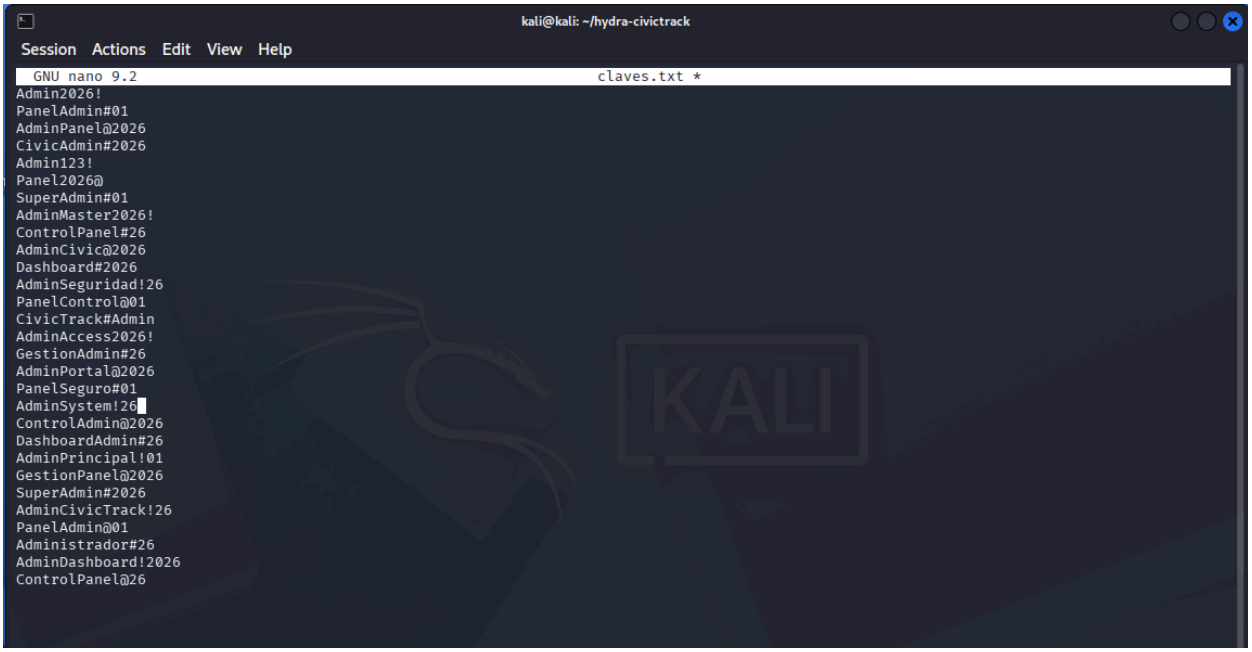
Dato	Valor confirmado
Endpoint	POST http://localhost:3000/api/auth/admin-login
Content-Type	application/json
Campos del body	{ "correo": "...", "password": "..." }
Fallo por correo inexistente	404 — {"message":"Usuario no encontrado"}
Fallo por contraseña incorrecta	401 Unauthorized — {"message":"Contraseña incorrecta"}
Éxito	200 con { user, token }

PASO 4: HYDRA: AUDITORÍA DEL MECANISMO DE AUTENTICACIÓN



Para la auditoría con hydra se crean dos documentos a los que se les hará la prueba, como se mencionó previamente, estos correos pertenecen al equipo de investigadores así que se tiene total autorización para usarlos; así mismo se crea un diccionario de claves para que hydra pueda

detectarlas, donde deliberadamente y solo para el taller se registró una clave real de una de las dos cuentas.



Diccionario de contraseñas:

Admin2026!
PanelAdmin#01
AdminPanel@2026
CivicAdmin#2026
Admin123!
Panel2026@
SuperAdmin#01
AdminMaster2026!
ControlPanel#26
AdminCivic@2026
Dashboard#2026
AdminSeguridad!26
PanelControl@01
CivicTrack#Admin
AdminAccess2026!
GestionAdmin#26
AdminPortal@2026

PanelSeguro#01

AdminSystem!26

ControlAdmin@2026

DashboardAdmin#26

AdminPrincipal!01

GestionPanel@2026

SuperAdmin#2026

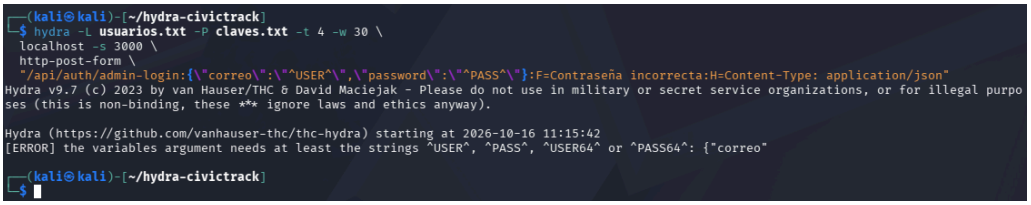
AdminCivicTrack!26

PanelAdmin@01

Administrador#26

AdminDashboard!2026

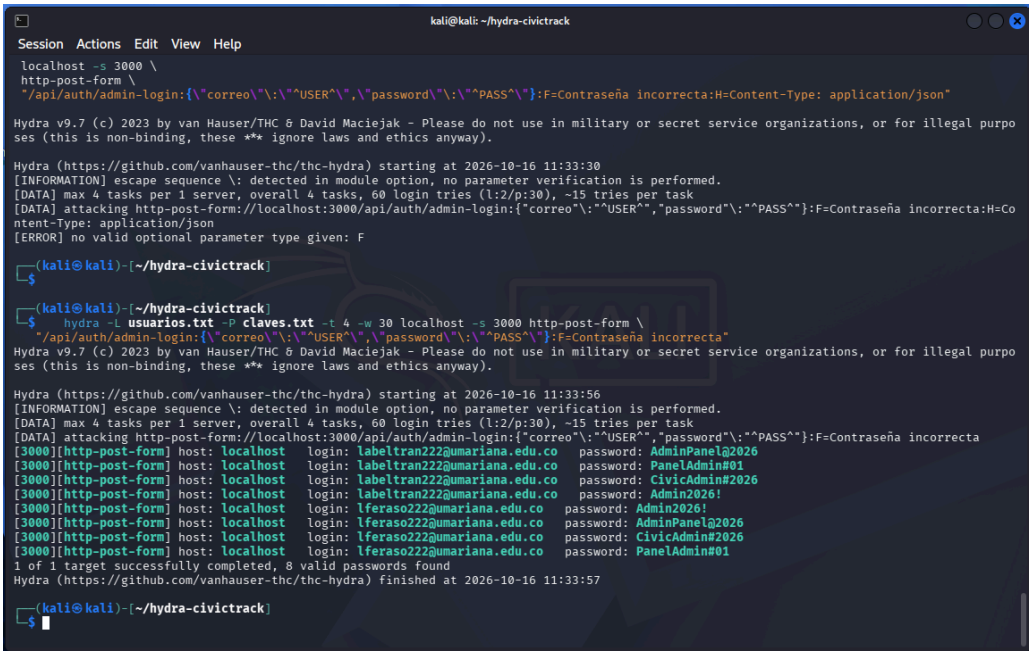
ControlPanel@26



Primer intento de Hydra: falla porque el JSON del body no fue reconocido, ya que Hydra usa : como separador interno y el cuerpo también lo usa sin escapar.



Segundo y tercer intento, ya con el JSON escapado (\:) pero con el header Content-Type todavía sin escapar, lo que sigue rompiendo el parser de Hydra.



Intento con el F= simplificado (sin la "ñ"): corre sin errores de sintaxis, pero marca las 8 combinaciones probadas como válidas para ambos usuarios — resultado descartado por no ser creíble (falso positivo, evidencia de que la condición de fallo tampoco se reconocía bien).

Explicación cambio de herramienta (Hydra por Script de Python): Durante la configuración de Hydra se realizaron varios intentos para adaptar el comando al login de CivicTrackIO, cuyo backend recibe las credenciales mediante JSON. Sin embargo, la sintaxis de http-post-form utiliza los dos puntos (:) como separadores internos, lo que generó conflictos con la estructura del JSON y con el encabezado Content-Type. Como resultado, Hydra no interpretaba correctamente algunos parámetros.

En uno de los intentos, Hydra ejecutó la prueba, pero el resultado no fue confiable: marcó como válidas todas las contraseñas, incluso cuando no correspondía. Esto indica que la condición utilizada para identificar un inicio de sesión fallido tampoco estaba siendo reconocida correctamente, posiblemente por el mensaje de respuesta utilizado como referencia, que contiene caracteres especiales como la ñ.

Ante esta limitación de Hydra, se optó por desarrollar un script en Python que reprodujera la misma prueba directamente sobre el endpoint /api/auth/admin-login. El script utiliza las listas usuarios.txt y claves.txt, envía cada combinación en formato JSON y analiza la respuesta del servidor para determinar si corresponde al mensaje de rechazo esperado.

Además, se mantuvieron condiciones de ejecución controladas: máximo cuatro solicitudes simultáneas, tiempo límite por petición y pausas entre tandas. De esta forma, la prueba permite evaluar las credenciales sin generar una carga innecesaria sobre el servidor.

Resultado:



¿Qué es lo que se encuentra?

El script probó las 60 combinaciones posibles (2 usuarios × 30 contraseñas) y, a diferencia del intento con Hydra, obtuvo un resultado coherente. Solo una combinación fue identificada como válida: labeltran222@umariana.edu.co con la contraseña adminlun123@, cuya respuesta fue HTTP 200, código que ya había sido confirmado como exitoso en el Paso 3.

Las 59 combinaciones restantes fueron rechazadas correctamente. Esto permite concluir que ninguna de las 30 contraseñas comunes probadas coincidió accidentalmente con la cuenta evaluada, lo que constituye un resultado favorable frente a un diccionario básico.

Además, la contraseña válida había sido incluida deliberadamente en la lista de prueba, siguiendo lo establecido en el Paso 4.1, para comprobar que el procedimiento fuera capaz de detectar una credencial válida. Por tanto, el único acierto corresponde a una credencial previamente conocida y no evidencia que el sistema pueda ser vulnerado fácilmente mediante contraseñas comunes.

Como aspecto de mejora, durante la prueba no se observó un mecanismo de bloqueo o limitación de intentos (rate limiting) ante múltiples autenticaciones fallidas. Aunque esto no permitió obtener credenciales adicionales, se recomienda considerar su implementación para fortalecer la protección del inicio de sesión.

Script de Python utilizado:

```
"""
Auditoria del mecanismo de autenticacion de CivicTrackIO (alternativa a Hydra).

Reproduce exactamente la peticion que ya se confirmo manualmente con curl
en el Paso 3 (POST JSON a /api/auth/admin-login), probando cada combinacion
usuario/clave de las wordlists ya creadas para el taller.

Respeta las mismas buenas practicas de la guia:
- Concurrencia limitada (equivalente al -t 4 de Hydra)
- Timeout por peticion (equivalente al -w 30 de Hydra)
- Pausa pequena entre tandas para no generar carga sostenida
- Un solo pase por el diccionario completo, no corridas repetidas

Uso:
   python3 bruteforce_login.py

Requiere: pip install requests --break-system-packages
"""

import itertools
import time
from concurrent.futures import ThreadPoolExecutor, as_completed

import requests

# ---- Configuracion (ajustar si algo cambio en el Paso 3) ----
URL = "http://localhost:3000/api/auth/admin-login"
USUARIOS_FILE = "usuarios.txt"
CLAVES_FILE = "claves.txt"
FAIL_STRING = "Contraseña incorrecta"  # texto exacto confirmado en el Paso 3
TIMEOUT = 30                    # segundos por peticion, igual que -w 30 en Hydra
MAX_WORKERS = 4                 # hilos concurrentes, igual que -t 4 en Hydra
DELAY_BETWEEN_BATCHES = 0.3     # segundos, para no generar rafagas tipo DoS


def cargar_lista(path):
   with open(path, encoding="utf-8") as f:
       return [linea.strip() for linea in f if linea.strip()]


def intentar_login(correo, password):
   try:
       r = requests.post(
```

```
            URL,
            json={"correo": correo, "password": password},
            timeout=TIMEOUT,
        )
    except requests.RequestException as e:
        return (correo, password, None, f"ERROR DE RED: {e}")

    if FAIL_STRING in r.text:
        return (correo, password, r.status_code, "fallo")
    else:
        return (correo, password, r.status_code, "POSIBLE EXITO")


def main():
    usuarios = cargar_lista(USUARIOS_FILE)
    claves = cargar_lista(CLAVES_FILE)
    combinaciones = list(itertools.product(usuarios, claves))

    print(f"[INFO] {len(usuarios)} usuarios x {len(claves)} claves = "
          f"{len(combinaciones)} intentos totales")
    print(f"[INFO] Objetivo: {URL}")
    print(f"[INFO] Condicion de fallo buscada: \"{FAIL_STRING}\"")
    print("-" * 70)

    encontrados = []
    errores = []

    # Se procesa en tandas de MAX_WORKERS para no disparar todo a la vez
    for i in range(0, len(combinaciones), MAX_WORKERS):
        tanda = combinaciones[i:i + MAX_WORKERS]
        with ThreadPoolExecutor(max_workers=MAX_WORKERS) as executor:
            futuros = [
                executor.submit(intentar_login, correo, password)
                for correo, password in tanda
            ]
            for futuro in as_completed(futuros):
                correo, password, status, resultado = futuro.result()
                if resultado == "POSIBLE EXITO":
                    print(f"[+] POSIBLE CREDENCIAL VALIDA -> {correo} : {password} "
                          f"(HTTP {status})")
                    encontrados.append((correo, password, status))
                elif resultado.startswith("ERROR DE RED"):
                    print(f"[!] {correo} : {password} -> {resultado}")
                    errores.append((correo, password, resultado))
                # los "fallo" normales no se imprimen para no saturar la salida
```

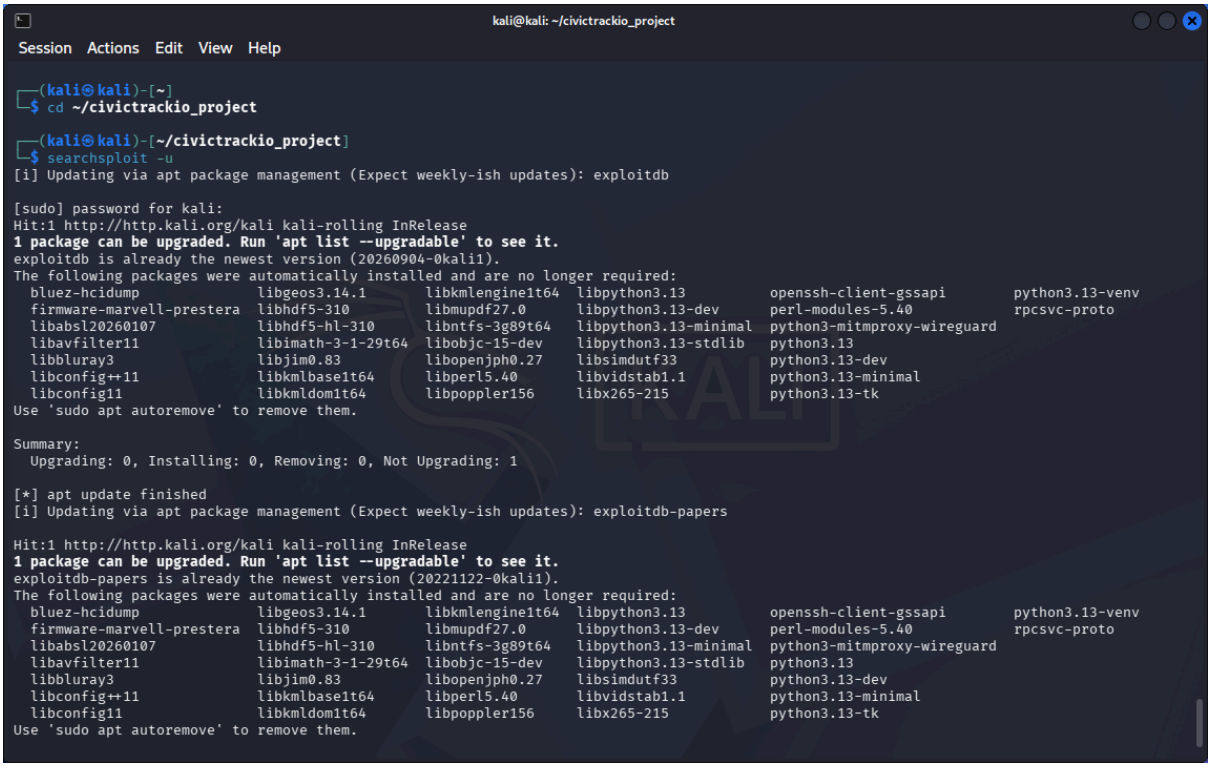
```
        time.sleep(DELAY_BETWEEN_BATCHES)


    print("-" * 70)
    print(f"[RESUMEN] Intentos totales: {len(combinaciones)}")
            print(f"[RESUMEN]    Posibles    credenciales    validas    encontradas:
{len(encontrados)}")
    for correo, password, status in encontrados:
        print(f"  {correo} : {password} (HTTP {status})")
    if errores:
        print(f"[RESUMEN] Errores de red/timeout: {len(errores)} "
              f"(si hay muchos, puede ser rate limiting del servidor: "
              f"tambien es un hallazgo valido)")

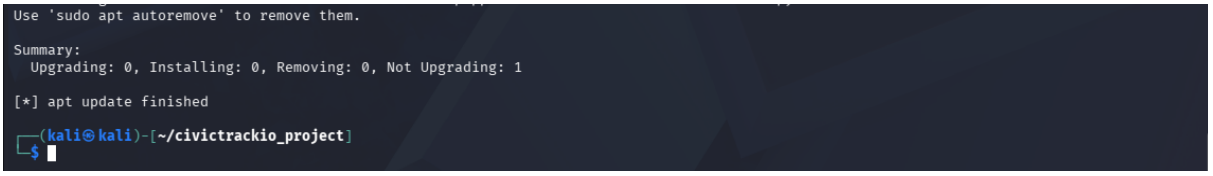


if __name__ == "__main__":
    main()
```

PASO 5: SEARCHSPLOIT: EXPLOITS CONOCIDOS POR TECNOLOGÍA/VERSIÓN



Se actualiza la base de datos local de Exploit-DB antes de realizar cualquier consulta.



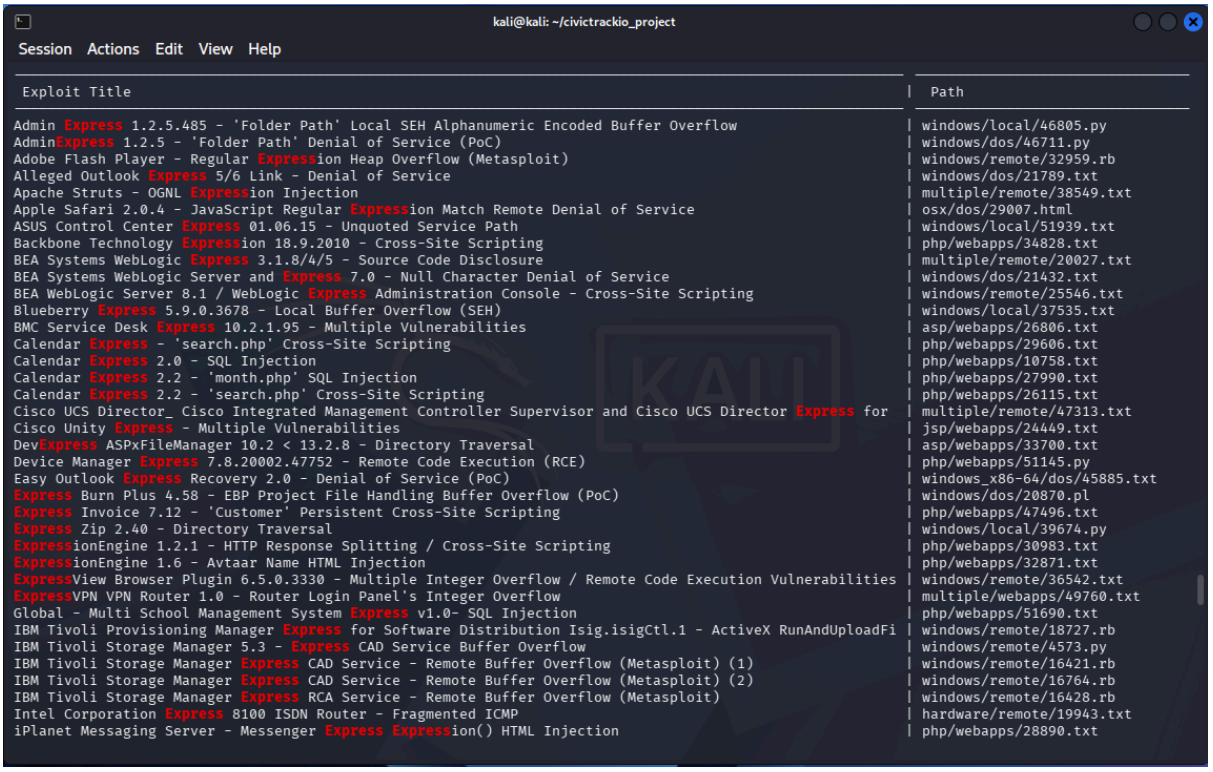
Confirmación de que la base de datos quedó actualizada.



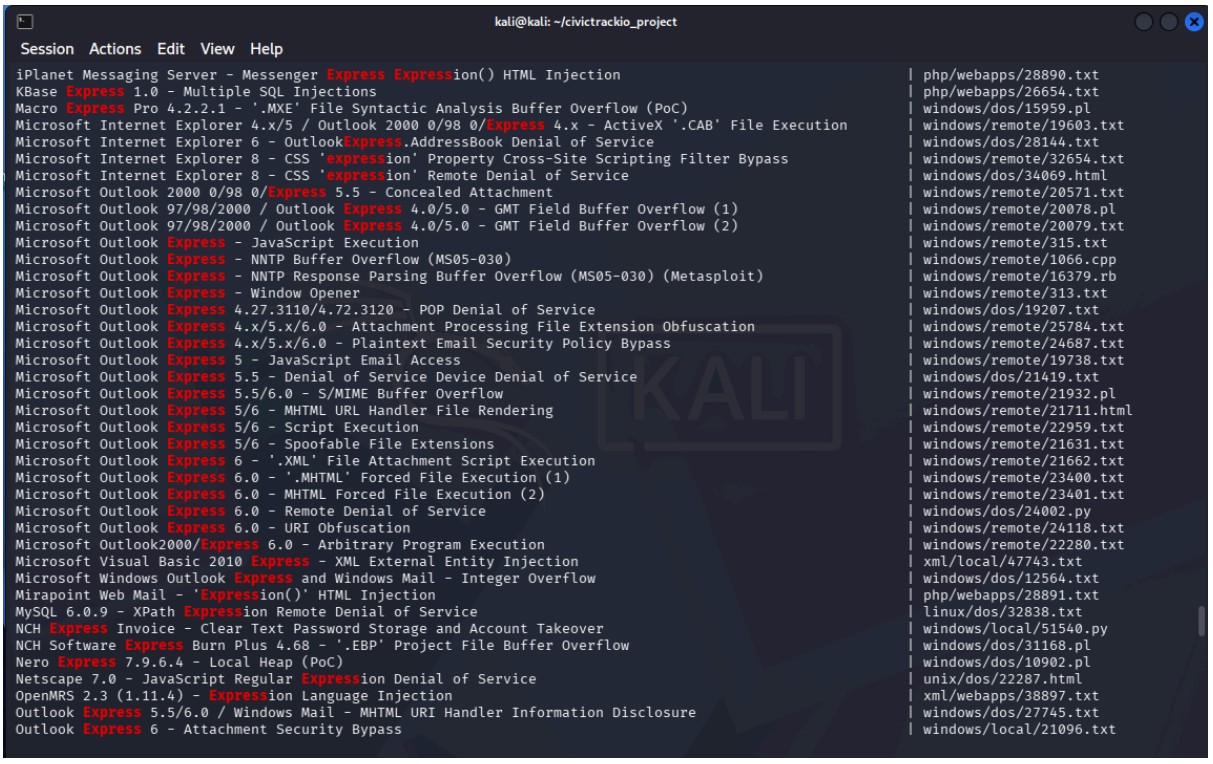
Se confirma nuevamente el header X-Powered-By: Express, ya visto en el Paso 2, esta vez de forma puntual y aislada.



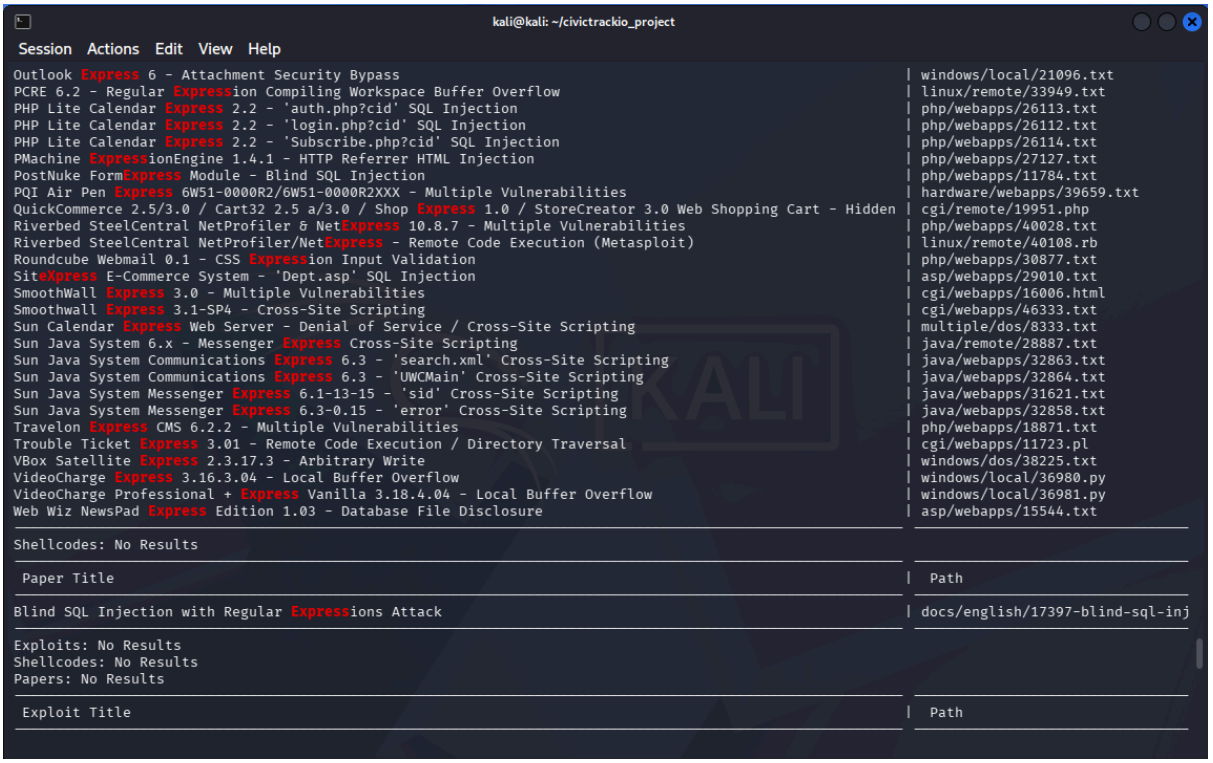
Se identifican las versiones reales de las dependencias del backend y frontend, y la versión de Node.js instalada (v24.21.0), insumos necesarios para las consultas en SearchSploit.



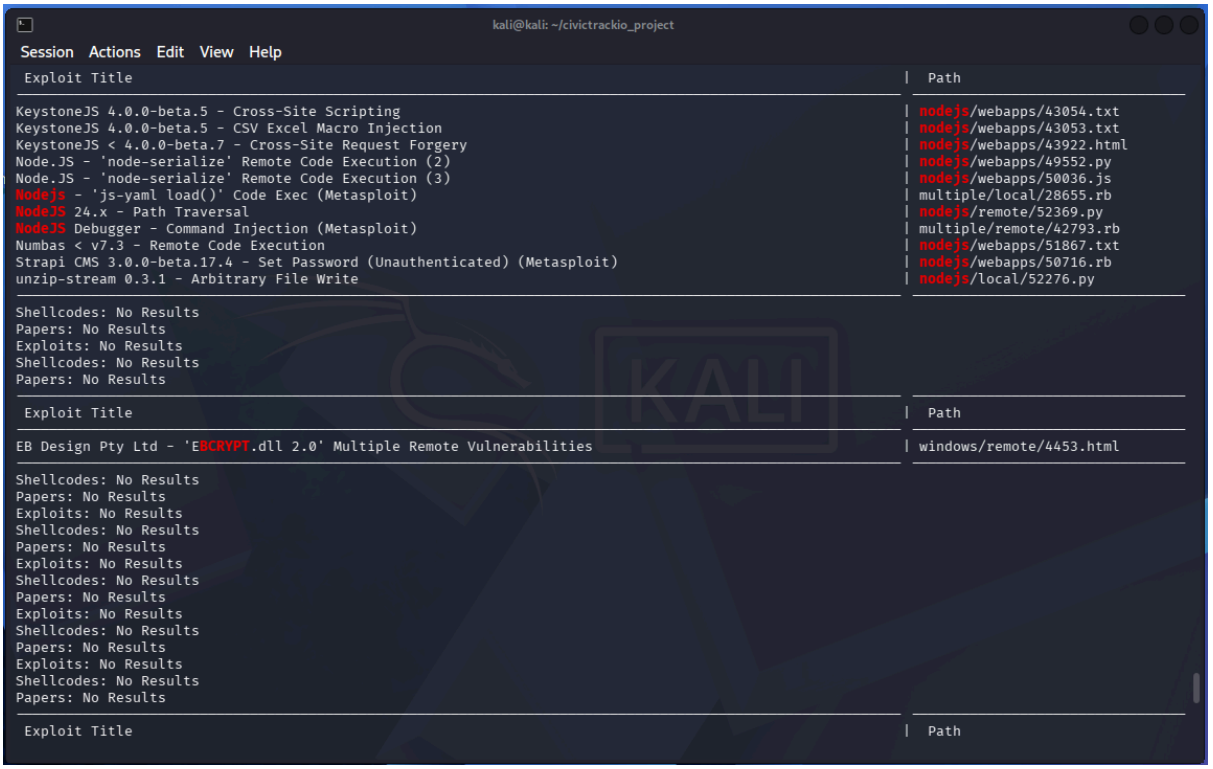
Resultados para "express": en su mayoría corresponden a Microsoft Outlook Express, no al framework.



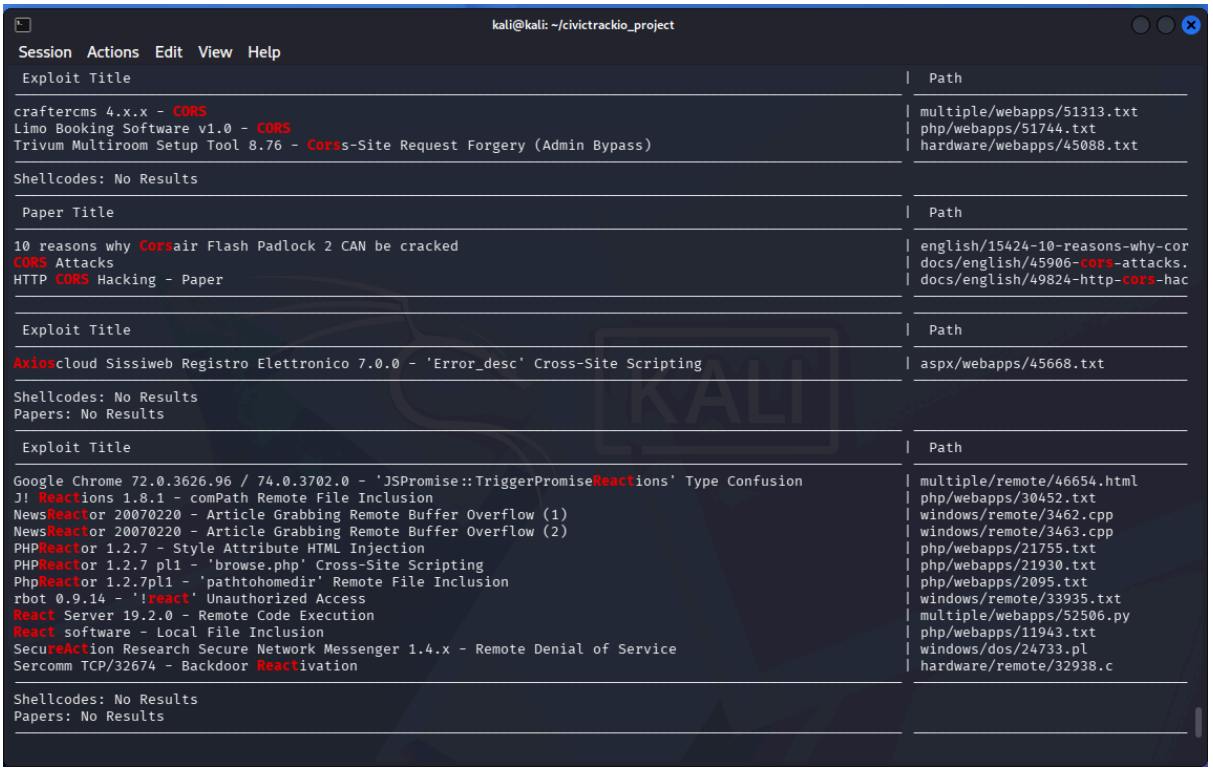
Continuación de resultados de Outlook Express, sin coincidencias reales.



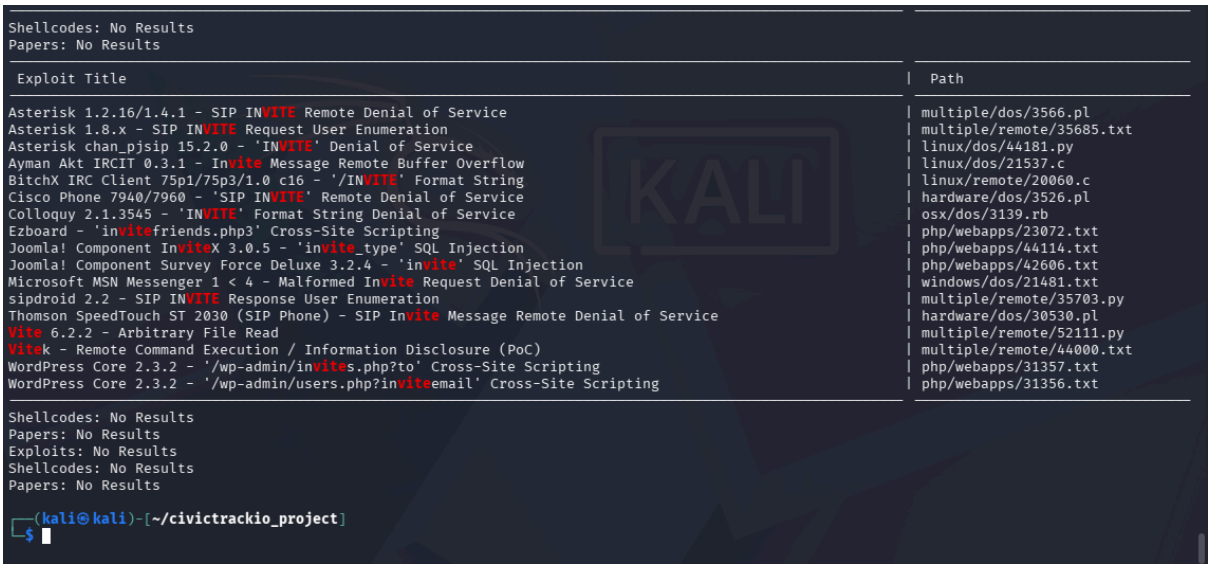
Cierre de resultados de “express”, confirmando que no hay coincidencias aplicables al proyecto.



Resultados para “nodejs”: se identifica NodeJS 24.x - Path Traversal, coincidencia exacta con la versión instalada, pendiente de verificar en detalle.



Sin coincidencias reales; los resultados corresponden a productos distintos que casualmente comparten el nombre con las librerías del proyecto.



Se identifica Vite 6.2.2 - Arbitrary File Read, versión cercana a la instalada (^6.3.5), pendiente de verificar en detalle.



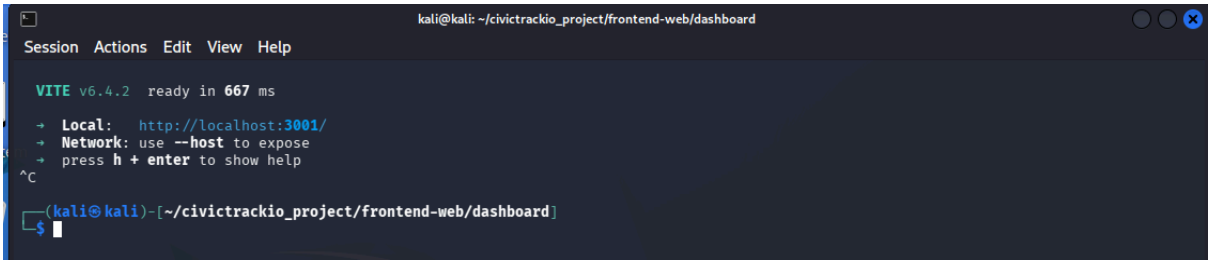
Se abre el detalle de ambos exploits, obteniendo sus CVE correspondientes (CVE-2025-27210 y CVE-2025-30208) para investigar las versiones exactas afectadas.

Tabla de hallazgos:

Tecnología / Versión	Resultado	Aplica al proyecto
Express ^5.2.1	Sin coincidencias reales en Exploit-DB	No
Node.js v24.21.0	NodeJS 24.x - Path Traversal (CVE-2025-27210)	No — ya parchado (fix en 24.4.1) y exclusivo de Windows
bcrypt, jsonwebtoken, @supabase/supabase-js, express-rate-limit, pg	Sin resultados en Exploit-DB	No
cors, axios, react	Sin coincidencias reales (nombres de producto distintos)	No
Vite ^6.3.5	Vite 6.2.2 - Arbitrary File Read (CVE-2025-30208)	No — ya parchado (fix en 6.2.3) y requiere exposición con --host, que no se usó

Finalmente se detienen los servicios y se apaga la vm:





Conclusiones.

1. **La infraestructura expuesta públicamente presenta una protección razonable, aunque todavía revela información innecesaria.** El uso de Cloudflare frente al dominio de Render aporta una capa adicional de protección; sin embargo, se identificaron detalles técnicos y configuraciones como el proveedor de hosting, la tecnología del backend y CORS abierto. También faltan algunas cabeceras de seguridad recomendadas, como HSTS, Permissions-Policy y Referrer-Policy.
2. **El mecanismo de autenticación resistió la prueba con contraseñas comunes.** De las 60 combinaciones evaluadas sobre dos cuentas administrativas, únicamente fue válida la credencial incluida deliberadamente para comprobar el funcionamiento de la prueba. Las demás fueron rechazadas, por lo que no se identificaron contraseñas triviales mediante el diccionario utilizado.
3. **No se evidenció un mecanismo efectivo de limitación de intentos.** Durante la prueba no se activó ningún bloqueo o *rate limiting*, a pesar de que el proyecto cuenta con la dependencia express-rate-limit. Esto puede indicar que la protección no está aplicada a la ruta de autenticación o que su configuración permite un número de intentos superior al evaluado. Se considera una mejora concreta y de bajo costo.
4. **Las tecnologías evaluadas no presentaron vulnerabilidades conocidas aplicables a la configuración actual.** Las coincidencias encontradas mediante SearchSploit para Node.js y Vite corresponden a vulnerabilidades que ya fueron corregidas en las versiones utilizadas. Además, la condición necesaria para explotar la vulnerabilidad asociada con Vite no está presente en la configuración evaluada.
5. **Las herramientas de pentesting pueden presentar limitaciones según la implementación evaluada.** La incompatibilidad encontrada con Hydra al trabajar con el login basado en JSON no invalidó la prueba. En su lugar, se documentó la limitación y se implementó un script equivalente en Python, permitiendo obtener resultados verificables sin depender de una herramienta específica.